



SocialMarkets

Desarrollo de Aplicaciones Multiplataforma

Trabajo fin de ciclo presentado por:	David de Antonio Palomar
Tutor/a:	Ramiro Martín Morro
Fecha:	13 de mayo de 2026

Departamento de Informática

Memoria del proyecto

Documento técnico

Índice

1. Introducción.....	3
1.1. Descripción y contexto del proyecto.....	3
1.2. Motivación del proyecto.....	4
1.3. Beneficios esperados.....	5
2. Objetivos.....	6
2.1. Objetivos generales.....	6
2.2. Objetivos específicos.....	7
3. Contexto actual.....	8
3.1. Estado del arte.....	8
3.2. Conceptos clave.....	10
4. Planificación.....	12
4.1. Metodología de gestión del proyecto.....	12
4.2. Temporización y secuenciación.....	14
4.3. Seguimiento.....	16
5. Análisis de requisitos.....	17
5.1. Diagrama de casos de uso.....	17
5.2. Requisitos funcionales/negocio.....	20
5.3. Requisitos no funcionales.....	22
5.4. Descripción de los usuarios y sus necesidades.....	24
6. Diseño de la aplicación.....	26
6.1. Mockups o wireframes o prototipos de la interfaz gráfica de usuario.....	26
6.2. Arquitectura del sistema.....	53
6.3. Diagramas Entidad/Relación.....	55
6.4. Diagramas Relacional.....	58
6.5. Diagrama de Clases.....	59
7. Desarrollo de la aplicación.....	61
7.1. Tecnologías y herramientas utilizadas.....	61
7.2. Descripción de las principales funcionalidades.....	64

8. Pruebas y Validación.....	70
8.1. Pruebas unitarias.....	70
8.2. Pruebas de integración.....	71
9. Despliegue y mantenimiento	72
9.1. Despliegue de la aplicación.....	72
10. Conclusiones y trabajo futuro.....	74
10.1 Conclusiones.....	74
10.1 Trabajo futuro.....	75
11. Relación del proyecto con los módulos del ciclo.....	76
12. Bibliografía/Webgrafía.....	78

1. Introducción

1.1. Descripción y contexto del proyecto

El proyecto SocialMarkets aparece con el objetivo de crear un espacio donde el conocimiento técnico y la interacción social se junten para ofrecer una experiencia financiera única. En su esencia, la plataforma es una terminal de análisis colaborativo que permite a los usuarios monitorizar el comportamiento de los mercados globales mientras participan en una comunidad de expertos y entusiastas de la inversión. A diferencia de las herramientas financieras tradicionales, que suelen ser frías y numéricas, SocialMarkets pone el foco en las personas, permitiendo que cada usuario construya su propia reputación como analista mediante la publicación de tesis detalladas y el seguimiento de su índice de acierto.

El escenario actual de las finanzas personales ha cambiado: el inversor de hoy busca algo más que una simple tabla de precios; busca contexto, debate y, sobre todo, fuentes de información en las que pueda confiar. En este contexto, el proyecto nace para combatir la desinformación y el ruido que a menudo inundan las redes sociales generales. Al integrar potentes herramientas de visualización en tiempo real con un sistema de clasificación basado en resultados reales, la plataforma consigue que los análisis de calidad ganen visibilidad por mérito propio. Así, SocialMarkets se convierte en un punto de encuentro donde el conocimiento se comparte de forma estructurada, permitiendo que tanto principiantes como expertos puedan aprender y crecer dentro de un entorno seguro y profesional.

Para dar vida a esta visión, se ha optado por un desarrollo tecnológico robusto y una interfaz de usuario sencilla de utilizar. La arquitectura se apoya en un backend sólido con Spring Boot, que gestiona de forma segura toda la lógica de usuarios y notificaciones, y un frontend dinámico construido con React, que destaca por una interfaz moderna y fluida. Cada detalle técnico se ha diseñado para que la herramienta sea intuitiva y atractiva. SocialMarkets aspira a ser el estándar para aquellos que entienden la inversión como una disciplina técnica que se enriquece cuando se comparte con inteligencia y rigor.

1.2. Motivación del proyecto

La elección de este proyecto nace de una mezcla entre un interés personal y una inquietud académica por explorar las intersecciones entre la tecnología y la economía. Siempre me ha resultado interesante cómo los mercados financieros actúan como un reflejo de la toma de decisiones humana y cómo el acceso a información de calidad puede transformar por completo el éxito de una inversión. Desarrollar SocialMarkets me ha permitido profundizar en estos ámbitos, aplicando mis conocimientos técnicos para crear una herramienta que no solo es útil, sino que también aporta claridad en un sector que a menudo resulta complejo para el inversor individual o principiante.

Desde el punto de vista profesional, mi principal motivación ha sido el reto de enfrentarme a un desarrollo *full-stack* de extremo a extremo, utilizando un stack tecnológico moderno y altamente demandado en el mercado laboral actual. La oportunidad de trabajar con Spring Boot para construir una lógica de negocio robusta, integrar servicios en la nube como AWS S3 y diseñar una interfaz dinámica con React, ha supuesto un aprendizaje constante. Este proyecto me ha obligado a salir de mi zona de confort, enfrentándome a desafíos reales como la seguridad mediante JWT, la gestión de datos relacionales complejos y la integración de APIs externas en tiempo real.

Finalmente, existe una motivación social y ética detrás de este trabajo. En un momento en el que el acceso a la inversión se ha facilitado, sentí la necesidad de proponer un sistema que premiara el rigor y el acierto sobre el simple ruido que puede llegar a ser desinformación o falta de conocimiento. Crear un sistema de clasificación basado en resultados reales no es solo un reto técnico de programación de algoritmos, sino una forma de aplicar la tecnología para fomentar un entorno de inversión más transparente y honesto. Esta combinación de pasión por las finanzas, compromiso con la calidad técnica y el deseo de mejora profesional constante es lo que me ha motivado a la realización de este Trabajo de Fin de Grado.

1.3. Beneficios esperados

La solución resultante de este proyecto reportará una serie de beneficios clave, estructurados tanto en el valor aportado al usuario final como en el crecimiento profesional del autor.

En lo que respecta al usuario final, la plataforma ofrece, en primer lugar, una centralización de herramientas financieras, permitiendo gestionar el análisis de activos, el seguimiento de gráficos en tiempo real y la interacción social en un único sistema, evitando la dispersión en diferentes aplicaciones o foros. Un beneficio es el acceso a un entorno de confianza y transparencia, donde gracias al sistema de ranking y las métricas de éxito, el usuario puede filtrar la información y fundamentar sus decisiones en recomendaciones de analistas con un rendimiento contrastado. Además, la aplicación proporciona una experiencia de usuario fluida y profesional, apoyada en una arquitectura escalable que garantiza la integridad de los datos y permite que la plataforma pueda crecer en el futuro con nuevas funcionalidades sin comprometer la estabilidad del sistema actual.

Por la parte del autor del proyecto, se identifican beneficios de gran impacto para su futuro profesional. El primero es la consolidación de competencias técnicas avanzadas, mediante el desarrollo de habilidades prácticas en tecnologías de alta demanda como Java, Spring Boot, React, PostgreSQL y AWS S3. Asimismo, el proyecto proporciona una valiosa experiencia en el diseño de arquitecturas profesionales, enfrentando retos reales como la implementación de seguridad mediante JWT y el uso de APIs externas de alto rendimiento. Finalmente, el resultado constituye un sistema completo que combina un backend robusto con un frontend moderno y estéticamente cuidado, sirviendo como una demostración sólida de la capacidad para integrar conocimientos adquiridos durante el grado en una solución real y funcional.

2. Objetivos

2.1. Objetivos generales

Los objetivos generales que se quieren alcanzar con el desarrollo de este proyecto son los siguientes:

- **Creación de un entorno colaborativo profesional:** Desarrollar una aplicación full-stack que permita a analistas e inversores publicar, seguir y evaluar proyecciones de mercado de forma estructurada, transformando el análisis individual en un activo compartido de valor para la comunidad.
- **Implementación de un sistema de reputación basado en datos:** Construir un motor de validación que contraste las predicciones de los usuarios con datos reales de mercado, generando un ranking de éxito que premie la precisión técnica y aporte transparencia a las interacciones dentro de la plataforma.
- **Optimización del proceso de toma de decisiones:** Proporcionar herramientas avanzadas de visualización y seguimiento en tiempo real (integración de terminales y notificaciones), que faciliten a los usuarios finales una toma de decisiones financieras más fundamentada y menos dependiente del ruido informativo externo.
- **Fomento del aprendizaje y la cultura financiera:** Promover un espacio de intercambio de conocimientos donde la participación activa y el análisis crítico ayuden a los usuarios, especialmente a estudiantes y principiantes, a mejorar sus competencias en el análisis de activos.
- **Excelencia técnica y usabilidad:** Aplicar estándares modernos de ingeniería de software para asegurar una arquitectura escalable y una interfaz de usuario (UI/UX) de alta calidad, que garantice una experiencia fluida, segura y atractiva para el usuario profesional.

2.2. Objetivos específicos

Para llegar a los objetivos generales, se establecen las siguientes metas específicas detalladas por áreas de desarrollo:

- **Análisis y Requisitos:** Identificar y definir con precisión los requisitos funcionales y no funcionales, asegurando que la plataforma responda tanto a las necesidades del usuario financiero como a los estándares de seguridad y rendimiento actuales.
- **Desarrollo del Backend:** Implementar una arquitectura de servidor robusta mediante una API REST en Spring Boot, asegurando una gestión eficiente de la lógica de negocio y la protección de rutas mediante autenticación JWT (JSON Web Tokens).
- **Interfaz de Usuario (Frontend):** Diseñar y construir una interfaz reactiva utilizando React y Vite, aplicando principios de *Glassmorphism* y animaciones fluidas para ofrecer una experiencia de usuario de alto nivel, intuitiva y profesional.
- **Gestión de Datos y Cloud:** Diseñar el modelo de datos relacional en PostgreSQL e integrar servicios de almacenamiento en la nube (AWS S3) para la gestión eficiente de imágenes de perfil y archivos multimedia de los analistas.
- **Lógica Financiera y Ranking:** Desarrollar el motor de cálculo del índice de acierto y el sistema de votaciones, garantizando que el procesamiento de los resultados de los análisis sea preciso y refleje objetivamente el desempeño de cada usuario.
- **Integración de Servicios Externos:** Incorporar módulos de visualización de gráficos en tiempo real mediante la API de TradingView, un sistema para obtener los precios en tiempo real mediante la API de Yahoo Finance y un sistema de noticias automatizado para contextualizar los análisis publicados.
- **Comunicación e Interacción:** Implementar un sistema de notificaciones asíncronas que mantenga a los usuarios informados sobre la actividad de la comunidad, nuevos seguidores y actualizaciones en los activos de su interés.
- **Validación y Documentación Técnica:** Realizar pruebas funcionales para asegurar la correcta inserción y persistencia de los datos en el sistema, validando la integridad de la información en el flujo completo de la aplicación. A su vez, elaborar una memoria técnica detallada que incluya el modelado de datos, diagramas de arquitectura y detalles de implementación, garantizando la trazabilidad y la futura escalabilidad del proyecto.

3. Contexto actual

3.1. Estado del arte

Este apartado se desarrolla analizando cuatro plataformas representativas del sector financiero y social actual, con el fin de contrastar sus enfoques y arquitecturas con la propuesta de este proyecto. Las aplicaciones analizadas son TradingView, Stocktwits, eToro e Investing.com:

- **TradingView** es la referencia mundial en gráficos y análisis técnico. Su núcleo es un motor gráfico extremadamente potente y una red social donde los usuarios comparten ideas. Su principal fortaleza es su especialización técnica y su enorme base de datos en tiempo real. Sin embargo, su red social es tan grande que a menudo resulta un caos. Es difícil para un usuario nuevo identificar quién es realmente un analista fiable, ya que la reputación se basa más en seguidores que en una tasa de acierto verificable y estructurada. Además, su interfaz puede resultar abrumadora para inversores menos experimentados.
- **Stocktwits** funciona como el "Twitter de la bolsa", centrándose exclusivamente en la inmediatez y el sentimiento del mercado. Destaca por su alta interactividad y por ser el punto de encuentro de millones de inversores minoristas. No obstante, su gran debilidad es la falta de rigor técnico. La información es volátil, no hay análisis profundos vinculados a precios objetivo reales y el ruido informativo (spam o predicciones sin fundamento) es muy elevado, lo que dificulta la toma de decisiones fundamentada.
- **eToro** es el pionero del "Social Trading". Su modelo permite a los usuarios copiar directamente las operaciones de otros inversores. Su ventaja es la transparencia en cuanto a las rentabilidades de los usuarios, ya que es un bróker cerrado. Sin embargo, su arquitectura es propietaria y está limitada exclusivamente a su plataforma de inversión. No permite el intercambio de tesis de inversión detalladas fuera de su ecosistema transaccional, y el enfoque es puramente financiero-especulativo, dejando de lado la parte educativa y de análisis técnico puro.

- **Investing.com** ofrece una gran cantidad de noticias y datos fundamentales. Es una herramienta de consulta masiva muy completa. Cuenta con una sección de comentarios en cada activo, pero carece de una verdadera estructura de comunidad de analistas. No hay perfiles profesionales donde se puedan seguir las métricas de éxito de un usuario a lo largo del tiempo, y su diseño suele estar saturado de publicidad, lo que penaliza la experiencia de usuario y la concentración necesaria para el análisis.

El análisis de estas soluciones revela una ventana de oportunidad clara para SocialMarkets. Mientras que las plataformas actuales o son demasiado técnicas (TradingView), demasiado ruidosas (Stocktwits) o demasiado cerradas (eToro), este proyecto propone una solución equilibrada: un backend como servicio independiente en Spring Boot que nutre a un cliente web ligero y moderno en React.

Esta arquitectura permite separar la lógica de cálculo de reputación de la visualización, ofreciendo al usuario una experiencia sobre todo, basada en la meritocracia.

3.2. Conceptos clave

Para la investigación, desarrollo y documentación de este proyecto, los conceptos fundamentales que definen su naturaleza y funcionamiento son los siguientes:

- **Arquitectura Cliente-Servidor (API REST):** Es el patrón arquitectónico sobre el que se asienta el sistema, donde un backend independiente desarrollado con Spring Boot gestiona la lógica de negocio y se comunica mediante el protocolo HTTP con un cliente ligero.
- **Consumo de APIs Financieras (Yahoo Finance):** Es el pilar de datos del proyecto. Se utiliza para la obtención de cotizaciones en tiempo real desde el servidor, permitiendo validar las predicciones de los usuarios y alimentar el motor de cálculo de resultados.
- **Terminal Financiera (TradingView):** Se refiere al módulo de visualización gráfica en el frontend. Permite el análisis técnico profesional mediante la integración de herramientas de dibujo y visualización de velas japonesas.
- **Social Trading / Análisis Colaborativo:** Concepto que define la interacción comunitaria en el ámbito de la inversión, donde los usuarios no solo consumen datos, sino que comparten tesis, predicciones y estrategias con otros miembros.
- **Ranking de Éxito (Algoritmo de Meritocracia):** Sistema diseñado para evaluar la fiabilidad de los analistas. Consiste en un motor de cálculo que mide el porcentaje de acierto de las predicciones financieras basándose en precios de mercado reales.
- **Single Page Application (SPA):** Tipo de aplicación web, desarrollada en este caso con React, que ofrece una experiencia de usuario fluida al cargar una única página y actualizar el contenido dinámicamente sin recargar el navegador.
- **Autenticación JWT (JSON Web Tokens):** Estándar de seguridad utilizado para la transmisión segura de información entre el cliente y el servidor, permitiendo la protección de rutas y la gestión de sesiones de usuario de forma escalable.

- **Cloud Storage (AWS S3):** Servicio de almacenamiento en la nube utilizado para la gestión persistente de archivos multimedia y fotos de perfil, permitiendo que el servidor de aplicaciones se mantenga ligero y eficiente.
- **Glassmorphism:** Estilo de diseño de interfaz de usuario caracterizado por el uso de transparencias, desenfoques de fondo y bordes sutiles, empleado en este proyecto para lograr una estética visual premium y moderna.

4. Planificación

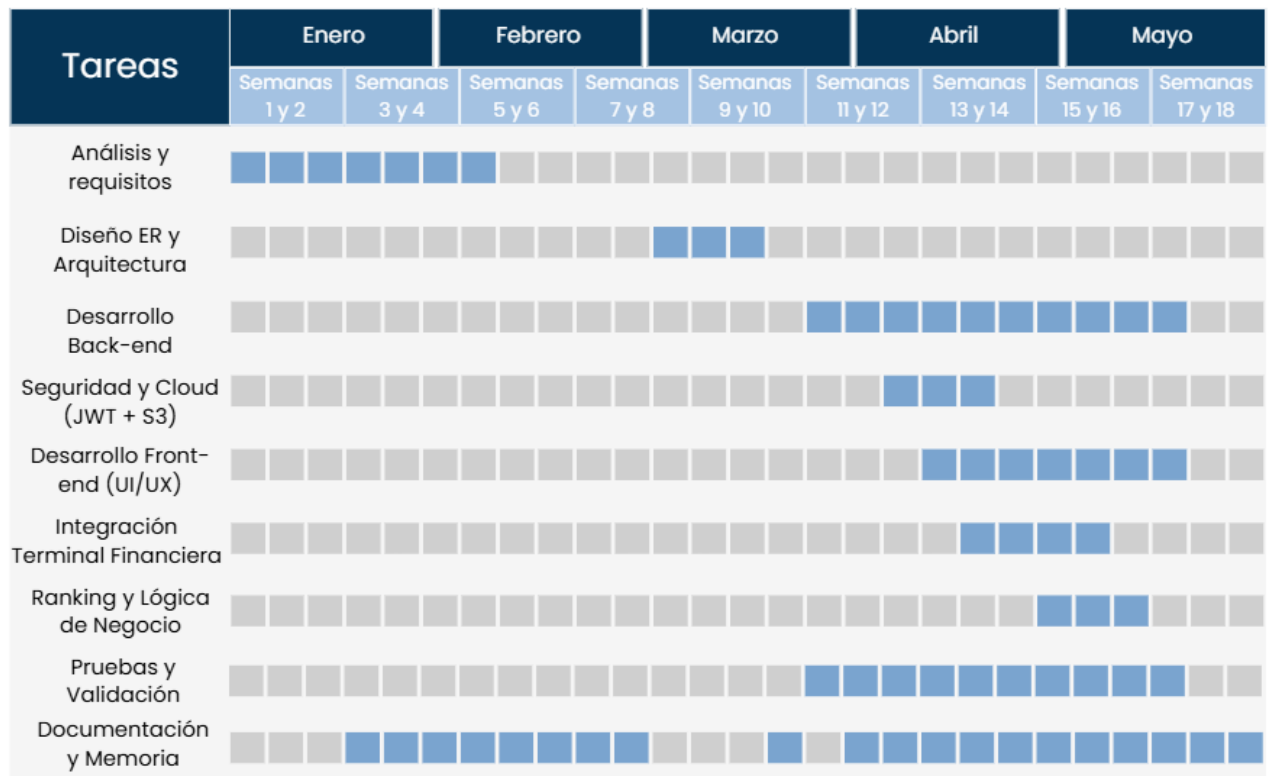
4.1. Metodología de gestión del proyecto

Para el desarrollo de **SocialMarkets**, se ha adoptado una metodología de trabajo ágil basada en el desarrollo iterativo e incremental. Al tratarse de un proyecto individual, se han adaptado los principios de marcos de trabajo como Scrum para maximizar la productividad y garantizar que cada funcionalidad sea testada y validada antes de pasar a la siguiente fase. Esta organización se apoya en los siguientes pilares:

- **Desarrollo por módulos funcionales:** En lugar de realizar un desarrollo lineal, el proyecto se ha dividido en bloques de funcionalidad independientes (Gestión de usuarios, Terminal financiera, Sistema de Ranking, etc.). Cada bloque se trata como una unidad de trabajo completa, lo que facilita la detección temprana de errores.
- **Control de versiones y flujo de trabajo:** Se utiliza GitHub como repositorio central y herramienta de control de software. Se ha seguido una política de commits estructurada (diferenciando entre correcciones, nuevas funcionalidades y documentación), lo que garantiza la trazabilidad total de los cambios. Este control de versiones permite trabajar de forma segura, pudiendo revertir cambios o experimentar con nuevas librerías sin comprometer la estabilidad de la rama principal.
- **Gestión del Backlog y Priorización:** Se mantiene un listado de tareas pendientes priorizadas según su criticidad técnica. Las tareas fundamentales, como la arquitectura del backend y la conexión con APIs externas, se abordaron en las fases iniciales para asegurar que los pilares del sistema fueran sólidos antes de avanzar en detalles estéticos o funcionalidades secundarias.
- **Documentación Evolutiva:** La redacción de la memoria técnica no se ha dejado para la fase final del proyecto, sino que se ha realizado de forma paralela al desarrollo del código. Este enfoque asegura que la documentación refleje con precisión las decisiones de diseño y los retos técnicos superados en el momento exacto en que ocurren.

- **Supervisión por Hitos Dirigidos:** El seguimiento del proyecto se ha articulado a través de una comunicación con el tutor asignado, quien ha ejercido un rol de guía mediante la asignación periódica de tareas organizadas. Este sistema ha permitido mantener un ritmo de trabajo constante, asegurando que cada fase del desarrollo cumpliera con los estándares de calidad requeridos y permitiendo una validación progresiva del avance del proyecto.

4.2. Temporización y secuenciación



1. **Análisis y requisitos:** Definición de casos de uso y estudio de APIs financieras.
2. **Diseño ER y Arquitectura:** Modelado de la base de datos PostgreSQL y diseño de la API REST.
3. **Desarrollo Back-end (API Core):** Implementación de servicios, controladores y lógica de Yahoo Finance.
4. **Seguridad y Cloud (JWT + S3):** Autenticación de usuarios e integración de almacenamiento en AWS.
5. **Desarrollo Front-end (UI/UX):** Estructura de la web en React, estilos CSS y navegación.
6. **Integración Terminal Financiera:** Implementación de gráficos de TradingView y datos en tiempo real.
7. **Ranking y Lógica de Negocio:** Algoritmo de cálculo de acierto y sistema de validación de analistas.
8. **Pruebas y Validación Funcional:** Pruebas de flujo de datos, inserciones y corrección de errores.

9. **Documentación y Memoria:** Redacción de los apartados de la memoria y manuales técnicos.

4.3. Seguimiento

El seguimiento del desarrollo de SocialMarkets se ha llevado a cabo mediante un control riguroso de las fases del ciclo de vida del software,

asegurando que cada objetivo técnico se completara antes de avanzar al siguiente. Para supervisar este progreso de forma efectiva, se han empleado los siguientes mecanismos:

- **Historial de Control de Versiones:** El repositorio en GitHub ha actuado como el diario técnico del proyecto. Gracias al uso de mensajes de commit descriptivos, se ha podido monitorizar la evolución diaria del código, permitiendo identificar el tiempo dedicado a cada funcionalidad y garantizando la integridad de las versiones del software.
- **Cumplimiento de Objetivos del Tutor:** El seguimiento se ha validado mediante la entrega progresiva de las tareas organizadas por el tutor. Cada bloque de trabajo completado servía como un punto de control para verificar que el desarrollo se alineaba con los objetivos planteados inicialmente.

5. Análisis de requisitos

1. **El actor usuario** representa al estudiante o analista que utiliza la interfaz de la plataforma para gestionar su perfil personal y publicar sus contenidos financieros, puede estar registrado o no.
2. **El actor sistema** actúa como el componente encargado de ejecutar las tareas automáticas y los cálculos de reputación sin necesidad de una acción directa del usuario.
3. **El actor API externa** de Yahoo Finance es el proveedor de información que suministra los valores de los activos y de TradingView, el listado de noticias a la aplicación y los gráficos.

Casos de Uso

4. **Iniciar sesión** permite al usuario identificarse en la plataforma para recibir un identificador de seguridad que le permitirá realizar acciones privadas.
5. **Gestionar cuenta** es el área donde el usuario controla su información personal permitiendo tanto crear un perfil nuevo como ajustar los datos del perfil actual.
6. **Registrarse** es la opción inicial para que los nuevos usuarios se incorporen a la plataforma creando su entrada correspondiente en la base de datos.
7. **Editar perfil** permite modificar la biografía y la imagen que el analista muestra al resto de la comunidad requiriendo siempre la validación de la identidad.
8. **Gestionar análisis** es el proceso central para crear proyecciones financieras que estarán vinculadas a datos de mercado reales y verificables.
9. **Seleccionar activo** obliga al usuario a elegir un instrumento financiero real obtenido directamente desde la fuente externa de yahoo finance.
10. **Seleccionar tipo de análisis** define la metodología que se ha seguido para la proyección y establece el plazo temporal estimado para la predicción.
11. **Agregar explicación** sirve para que el usuario detalle los argumentos que respaldan su postura financiera para informar mejor al resto de la comunidad.

12. **Publicar análisis** guarda la información definitiva en el sistema incluyendo siempre una validación del identificador de sesión para asegurar la autoría.
13. **Interactuar con comunidad** engloba las actividades sociales que fomentan la participación y la validación mutua entre los miembros de la red.
14. **Votar análisis** permite calificar de forma positiva o negativa las publicaciones de otros miembros para ayudar a filtrar el contenido de mayor calidad.
15. **Seguir analista** establece una relación de interés entre perfiles para que un usuario pueda estar al tanto de las novedades de sus autores favoritos.
16. **Validar token** es el mecanismo técnico de seguridad que asegura que cada petición que modifica datos proviene de un usuario correctamente identificado.
17. **Consultar datos** es la función de lectura que muestra los gráficos de precios y las noticias actuales para que el usuario analice el mercado antes de decidir.
18. **Sincronizar noticias** es una tarea automática del sistema que recoge las novedades de TradingView para mantener el muro informativo actualizado sin usar inteligencia artificial.
19. **Resolver proyecciones** es el proceso mediante el cual el sistema verifica si los análisis han sido acertados comparando los precios para actualizar la reputación del autor.

5.2. Requisitos funcionales/negocio

El sistema se divide en cinco módulos principales que agrupan las funcionalidades necesarias para ofrecer una experiencia completa de análisis financiero colaborativo:

1. Módulo de Gestión de Usuarios y Seguridad

- **Registrar nuevo analista:** Permitir que los usuarios creen una cuenta proporcionando un nombre de usuario, correo electrónico y contraseña.
- **Autenticar usuario:** Validar las credenciales para permitir el acceso seguro a las funciones privadas de la plataforma.
- **Gestionar perfil profesional:** Permitir al usuario actualizar su biografía, nombre de usuario y fotografía de perfil (almacenada en la nube).
- **Finalizar sesión:** Desvincular de forma segura la cuenta del usuario del navegador actual.

2. Módulo de Terminal Financiera y Mercados

- **Consultar gráficos en tiempo real:** Visualizar la evolución del precio de activos mediante gráficos técnicos interactivos.
- **Buscar activos financieros:** Localizar acciones, criptomonedas o divisas mediante un buscador inteligente por nombre o ticker.
- **Visualizar cinta de precios (Ticker Tape):** Mostrar una barra animada con las últimas cotizaciones de los mercados globales en la parte superior de la aplicación.

3. Módulo de Análisis y Comunidad

- **Publicar tesis de inversión:** Permitir que los analistas redacten análisis detallados incluyendo el activo, el precio objetivo, el nivel de riesgo y su justificación técnica.
- **Interactuar con análisis:** Ofrecer la posibilidad de votar o reaccionar a las publicaciones de otros miembros de la comunidad.
- **Seguir a otros analistas:** Permitir que los usuarios se sigan entre sí para recibir actualizaciones sobre sus nuevas publicaciones.

- **Explorar el Feed de noticias:** Visualizar un muro con los últimos análisis publicados automáticamente desde la API de TradingView, de forma cronológica.

4. Módulo de Ranking y Reputación

- **Calcular índice de éxito:** Evaluar automáticamente la precisión de las predicciones de cada analista basándose en los precios reales de mercado.
- **Consultar clasificación (Leaderboard):** Visualizar una lista ordenada de los mejores analistas de la plataforma según su rendimiento y tasa de acierto.
- **Visualizar métricas de rendimiento:** Mostrar en el perfil de cada usuario sus estadísticas de éxito, número de seguidores y volumen de predicciones realizadas.

5. Módulo de Información y Notificaciones

- **Recibir notificaciones de actividad:** Alertar al usuario en tiempo real cuando alguien comienza a seguirle.
- **Consultar noticias del sector:** Ofrecer un flujo constante de noticias financieras relevantes para contextualizar el análisis de los mercados.

5.3. Requisitos no funcionales

1. Requisitos técnicos

- **Tecnología de Servidor:** El backend debe estar desarrollado sobre el ecosistema Java utilizando Spring Boot 4.0 para garantizar una arquitectura robusta y escalable.
- **Tecnología de Cliente:** La interfaz web debe construirse con React 19 bajo el empaquetador Vite, asegurando una carga rápida y una reactividad eficiente de los componentes.
- **Persistencia de Datos:** El sistema debe utilizar PostgreSQL 16 como sistema gestor de base de datos relacional para garantizar la integridad y consistencia de la información financiera y de usuario.
- **Integración en la Nube:** La gestión de archivos multimedia y fotografías de perfil debe realizarse mediante el servicio AWS S3, utilizando buckets privados para el almacenamiento persistente.
- **Consumo de APIs Externas:** La plataforma debe ser capaz de integrar y procesar datos de terceros mediante conexiones seguras a las APIs de TradingView (gráficos, ticker tape y noticias) y Yahoo Finance (precios en tiempo real).
- **Control de Software:** El seguimiento del código fuente debe gestionarse mediante Git, utilizando GitHub como repositorio centralizado.

2. Requisitos de rendimiento

- **Tiempo de Respuesta:** Las peticiones a la API para operaciones básicas (como cargar el perfil o publicar un análisis) deben tener un tiempo de respuesta medio inferior a 1.5 segundos en condiciones normales.
- **Carga de Gráficos:** Los módulos visuales de TradingView deben cargarse de forma asíncrona para no bloquear la interacción del usuario con el resto de la interfaz.
- **Optimización de Recursos:** El cliente web debe estar optimizado para mantener un consumo moderado de memoria RAM, incluso durante la visualización de múltiples activos financieros en tiempo real.

3. Requisitos de seguridad

- **Protección de Identidad:** El acceso a las funcionalidades privadas debe estar protegido mediante tokens JWT (JSON Web Tokens) con tiempos de expiración controlados.
- **Seguridad de Contraseñas:** Las claves de acceso de los usuarios no deben almacenarse en texto plano, sino mediante algoritmos de cifrado unidireccional como BCrypt.
- **Gestión de Credenciales:** Las claves sensibles (AWS, JWT secret, DB password) deben gestionarse mediante variables de entorno o archivos de configuración protegidos, nunca directamente en el código fuente público.
- **Comunicación Segura:** Todas las peticiones entre el cliente y el servidor deben estar preparadas para viajar bajo el protocolo HTTPS.

4. Requisitos de usabilidad

- **Diseño Estético (UI):** La interfaz debe seguir una estética moderna basada en Glassmorphism, utilizando transparencias y una paleta de colores coherente con un entorno financiero premium.
- **Feedback al Usuario:** El sistema debe proporcionar confirmaciones visuales inmediatas tras cada acción relevante del usuario, como el registro con éxito o la publicación de un análisis.
- **Navegación Intuitiva:** La estructura de la aplicación debe permitir al usuario acceder a cualquier sección principal (Mercados, Ranking, Perfil) en menos de dos clics desde el menú lateral.

5. Requisitos de disponibilidad

- **Acceso Continuo:** El sistema debe estar diseñado para permitir el acceso multiusuario simultáneo, garantizando que los datos de mercado estén actualizados durante las horas de apertura de los mercados globales.
- **Tolerancia a Fallos:** En caso de caída de una API externa (como Yahoo Finance), el sistema debe mostrar mensajes informativos adecuados y permitir el uso del resto de funcionalidades que no dependan de esos datos en tiempo real.

5.4. Descripción de los usuarios y sus necesidades

El sistema está diseñado para dar respuesta a diferentes perfiles de usuarios que, aunque comparten el interés por los mercados financieros, tienen objetivos y necesidades distintas dentro de la plataforma. Los perfiles principales identificados son:

- **Analista / Inversor Experimentado:** Es un usuario con conocimientos técnicos que busca un escaparate profesional para sus estrategias. Sus necesidades principales son:
 - **Herramientas de precisión:** Requiere gráficos de alta calidad y datos en tiempo real para fundamentar sus tesis de inversión.
 - **Construcción de reputación:** Necesita que sus aciertos sean registrados de forma objetiva y automática para ganar visibilidad y autoridad ante la comunidad.
 - **Portfolio personal:** Busca disponer de un perfil profesional donde pueda mostrar su histórico de análisis y métricas de rendimiento a otros usuarios. Esto puede ser útil si un inversor quiere entrar a trabajar a un fondo de inversión y le piden experiencia o resultados previos.
- **Inversor Principiante o Estudiante:** Es un usuario que está en fase de aprendizaje y busca fuentes de información fiables y herramientas accesibles. Sus necesidades son:
 - **Filtrado de calidad:** Necesita distinguir fácilmente entre un análisis riguroso y una opinión sin fundamento, apoyándose en el sistema de ranking de la plataforma.
 - **Centralización de información:** Requiere una interfaz que unifique noticias, gráficos y comunidad sin necesidad de navegar por múltiples webs externas.
 - **Seguimiento de referentes:** Necesita poder seguir y recibir actualizaciones de analistas con una alta tasa de acierto contrastada por el sistema.

SocialMarkets satisface estas necesidades mediante una arquitectura que combina la potencia de una terminal financiera profesional con la agilidad de una red social moderna. Gracias al motor de ranking y a la integración de datos en tiempo real, la plataforma ofrece un entorno donde la transparencia y la utilidad técnica se juntan para potenciar el crecimiento de todos sus usuarios, ofreciendo una experiencia unificada y centrada en el valor del análisis bien ejecutado.

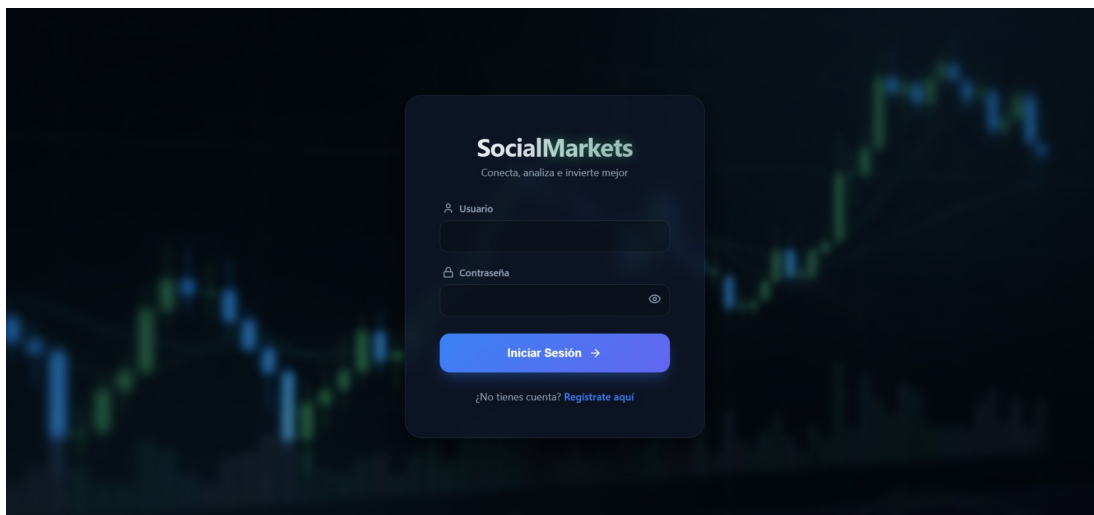
6. Diseño de la aplicación

6.1. Mockups, wireframes o prototipos de la interfaz gráfica de usuario

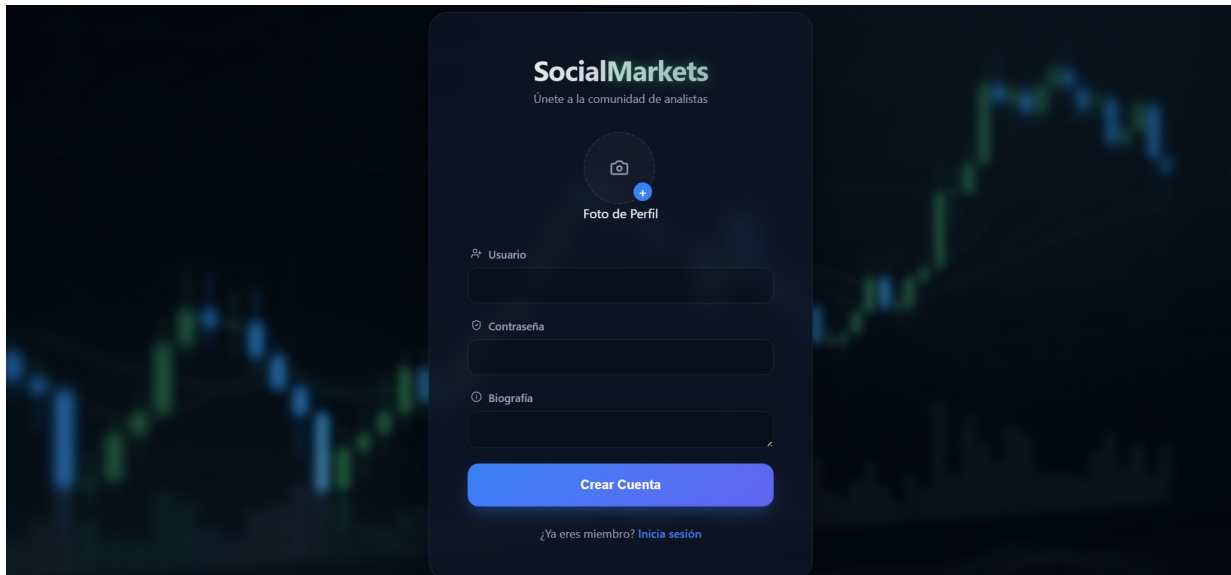
En este apartado se detalla el aspecto visual y funcional definitivo de SocialMarkets. Las siguientes capturas de pantalla representan la implementación real de la plataforma. El objetivo principal ha sido crear una "Terminal Social" que combine una estética premium basada en el *Glassmorphism* con una arquitectura de información eficiente, permitiendo que los datos financieros complejos se presenten de forma clara y atractiva.

La interfaz se ha estructurado mediante un sistema de componentes reactivos que aseguran una navegación fluida y una respuesta inmediata a las acciones del usuario. Se ha prestado especial atención a la coherencia visual, el uso de tipografías modernas y la adaptabilidad de los elementos, logrando un entorno de trabajo que reduce la carga cognitiva del inversor y fomenta la interacción comunitaria. A continuación, se describen las vistas principales que conforman el ecosistema de la aplicación y algunas de sus funciones:

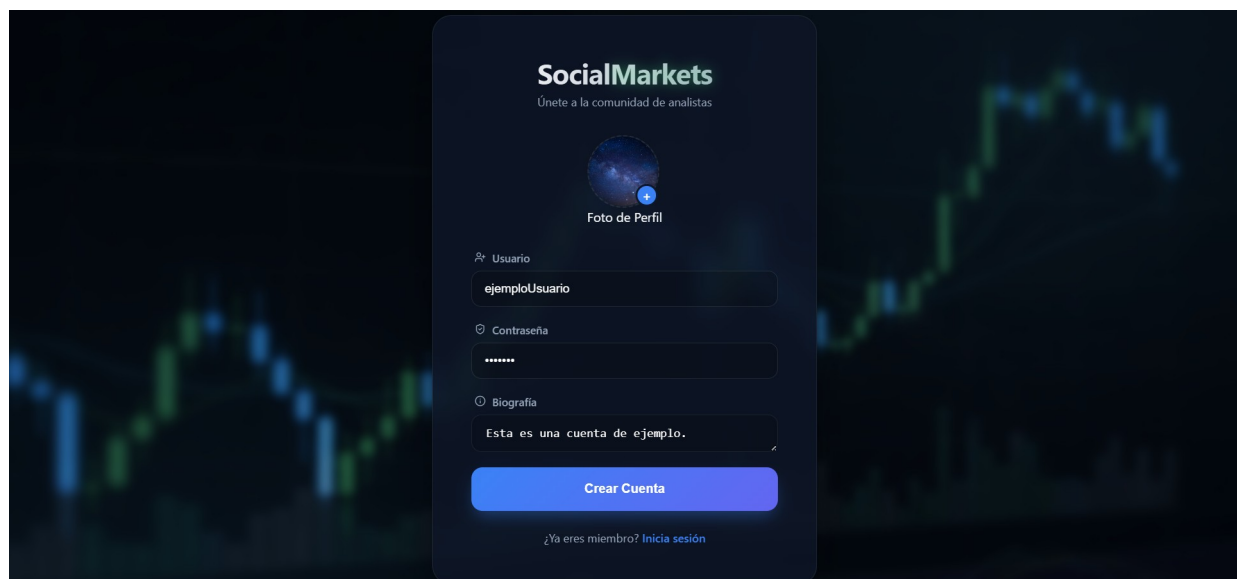
Nada más acceder, el usuario se encontrará con una pantalla de inicio de sesión para que el usuario entre si ya dispone de una cuenta. Si al contrario no tiene cuenta, podrá pulsar el botón "Regístrate aquí" para que le llegue a la pantalla de registro.



Lo que se ve es la pantalla de registro, con una foto de perfil opcionales, y un usuario y contraseña obligatorios. Además habrá un botón para volver al login en caso de que el usuario sí que disponga de una cuenta.

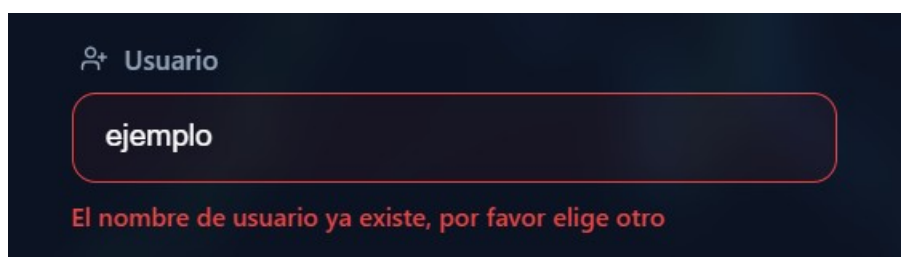


The screenshot shows the SocialMarkets registration interface. At the top, the logo 'SocialMarkets' is displayed with the tagline 'Únete a la comunidad de analistas'. Below this is a circular profile picture placeholder with a camera icon and a plus sign, labeled 'Foto de Perfil'. The form consists of three input fields: 'Usuario' (User), 'Contraseña' (Password), and 'Biografía' (Bio). The 'Usuario' field contains the text 'ejemploUsuario'. Below the fields is a blue 'Crear Cuenta' (Create Account) button. At the bottom, there is a link that says '¿Ya eres miembro? Inicia sesión' (Already a member? Log in).



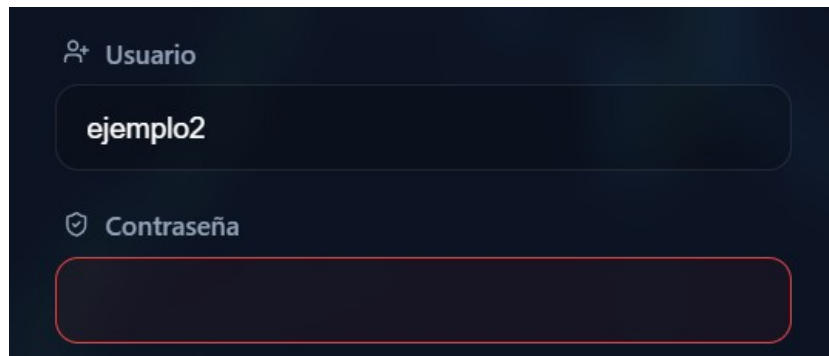
This screenshot shows the same registration form as above, but with a red error message. The 'Usuario' field now contains the text 'ejemplo'. Below the 'Biografía' field, a red message reads: 'El nombre de usuario ya existe, por favor elige otro' (The username already exists, please choose another). The 'Crear Cuenta' button and the '¿Ya eres miembro? Inicia sesión' link remain visible at the bottom.

Si el usuario elige un nombre de usuario existente se le notificará a través de un mensaje de error en rojo:



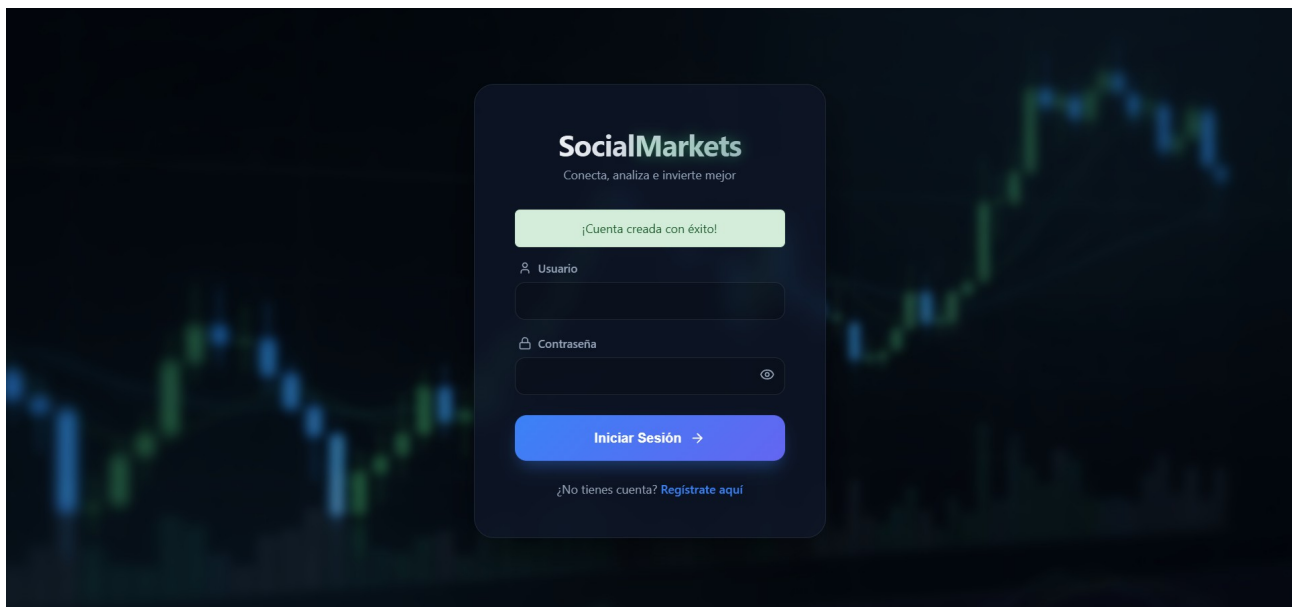
This is a close-up of the error message area. It shows the 'Usuario' label and the input field containing 'ejemplo'. Below the field, the red error message is clearly visible: 'El nombre de usuario ya existe, por favor elige otro'.

Si hay algún campo obligatorio faltante, tanto como en la página de registro como en el login, el cuadrado faltante se pondrá rojo:



A dark-themed login form. At the top, there is a label 'Usuario' with a person icon. Below it is a text input field containing 'ejemplo2'. Below that is a label 'Contraseña' with a shield icon. The password input field is empty and has a red border, indicating it is a required field that is currently empty.

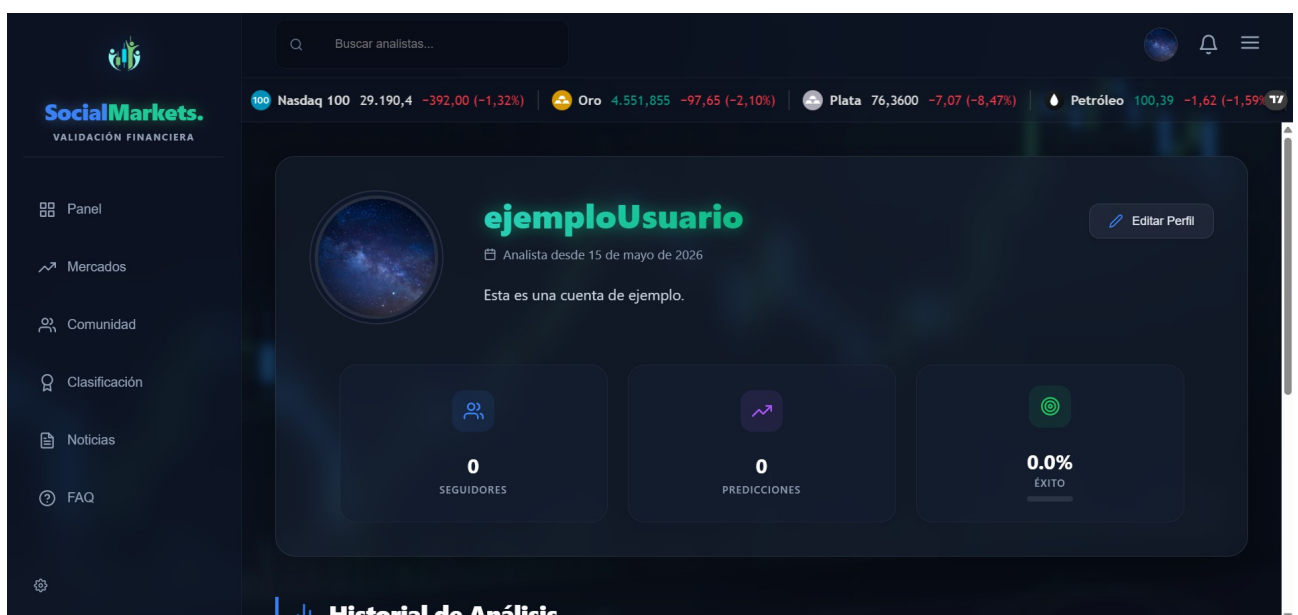
Lo que verá el usuario al registrarse será una redirección al login y un mensaje de éxito:



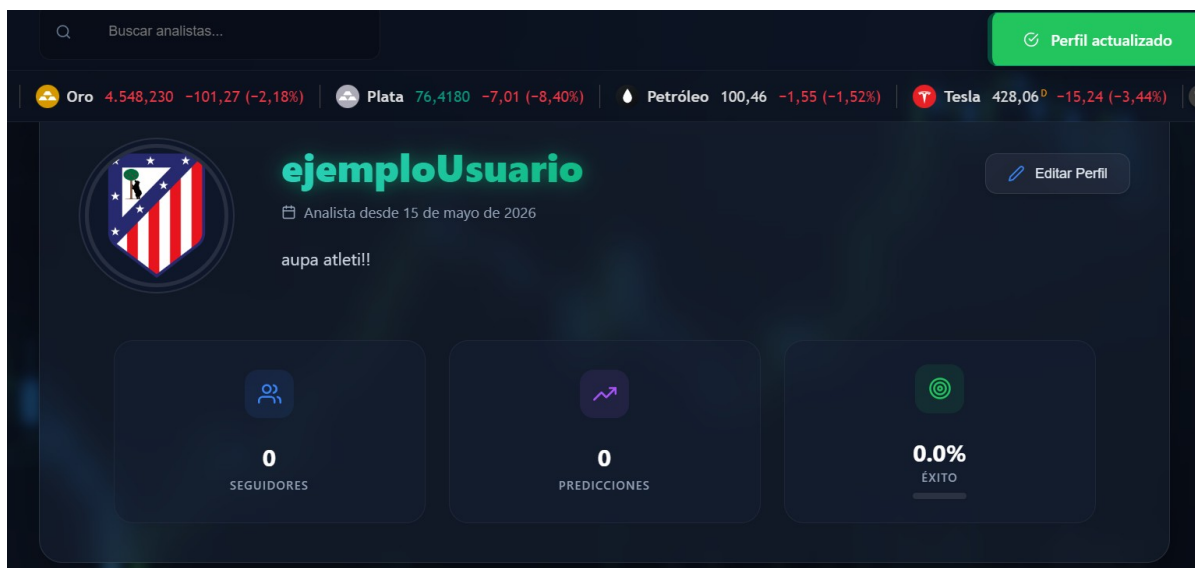
Al iniciar sesión, lo primero que verá el usuario será el panel principal, con sus estadísticas (todo a 0 si es una cuenta nueva), las diferentes páginas de navegación a la izquierda, un buscador de analistas en la barra superior, su foto de perfil, notificaciones y menú arriba a la derecha, la ticker tape con la cotización de los activos más conocidos, un botón para publicar un nuevo análisis (que redirige a la página de comunidad dentro del formulario de publicación de nuevo análisis) y un engranaje abajo a la izquierda para entrar en la página de ajustes:



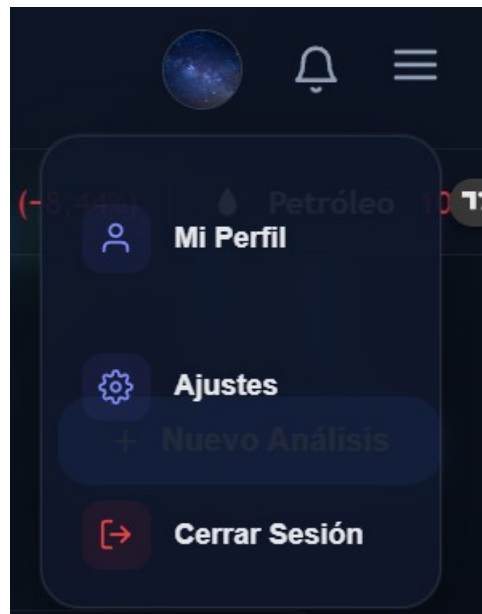
Si el usuario pulsa en su foto de perfil podrá acceder a su perfil de analista que verá el resto de la comunidad:



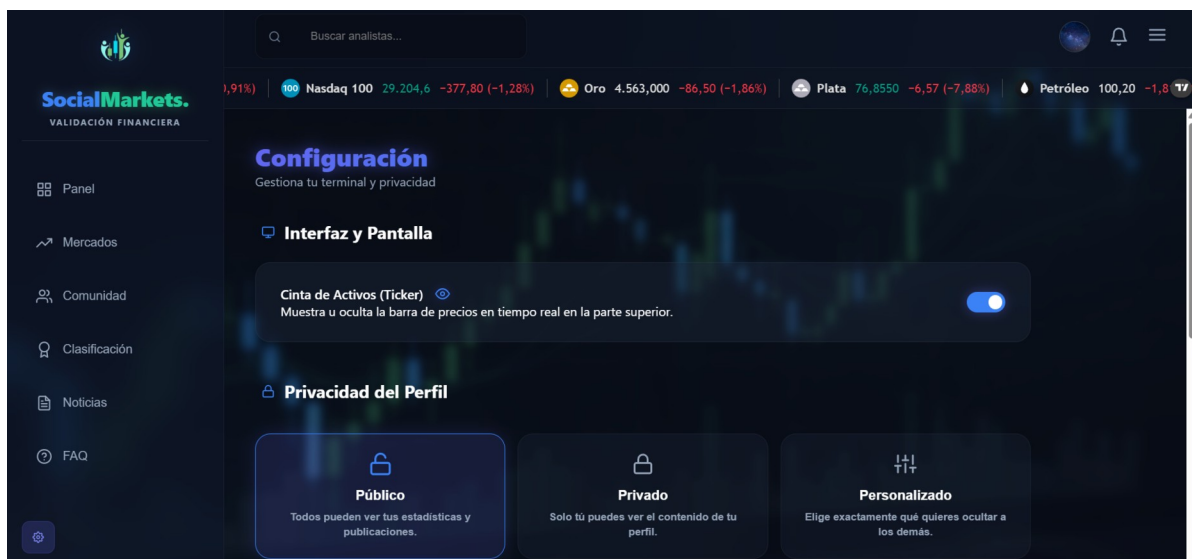
El usuario podrá editar su foto de perfil y descripción y guardar los cambios:

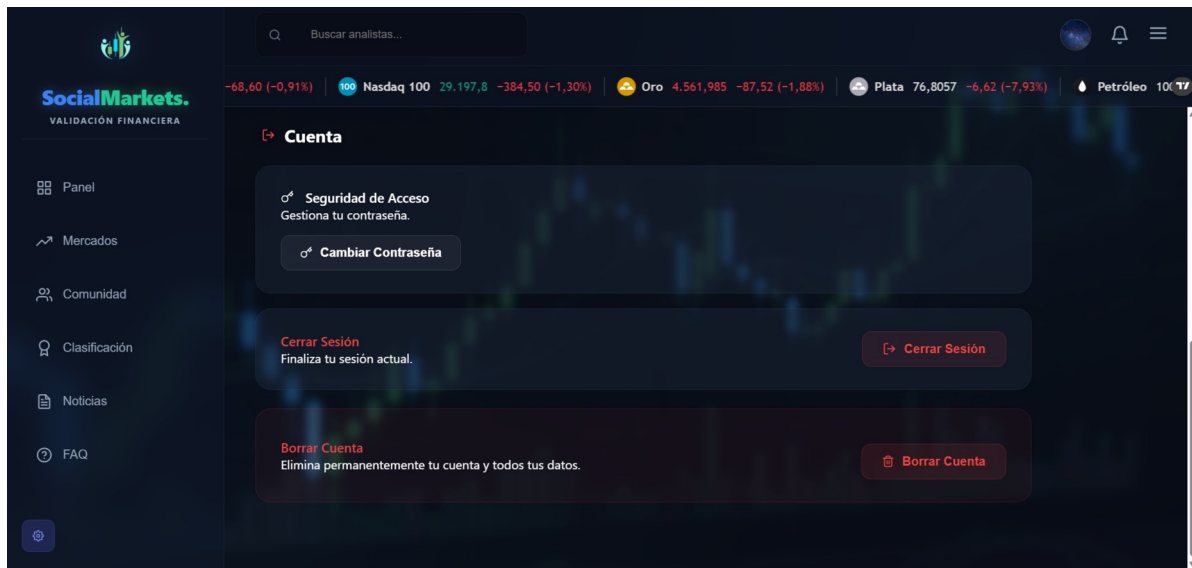


Al pulsar en las 3 rayas, aparecerá un menú con 3 opciones, ir al perfil (misma función que pulsar en la foto de perfil), la página de ajustes, y el cierre de sesión:

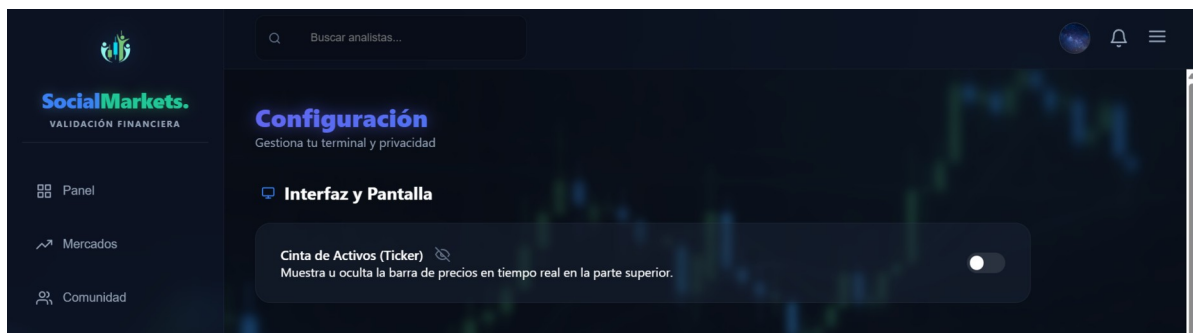


Al pulsar en ajustes se llevará a la siguiente página (también accesible desde el menú lateral, abajo):

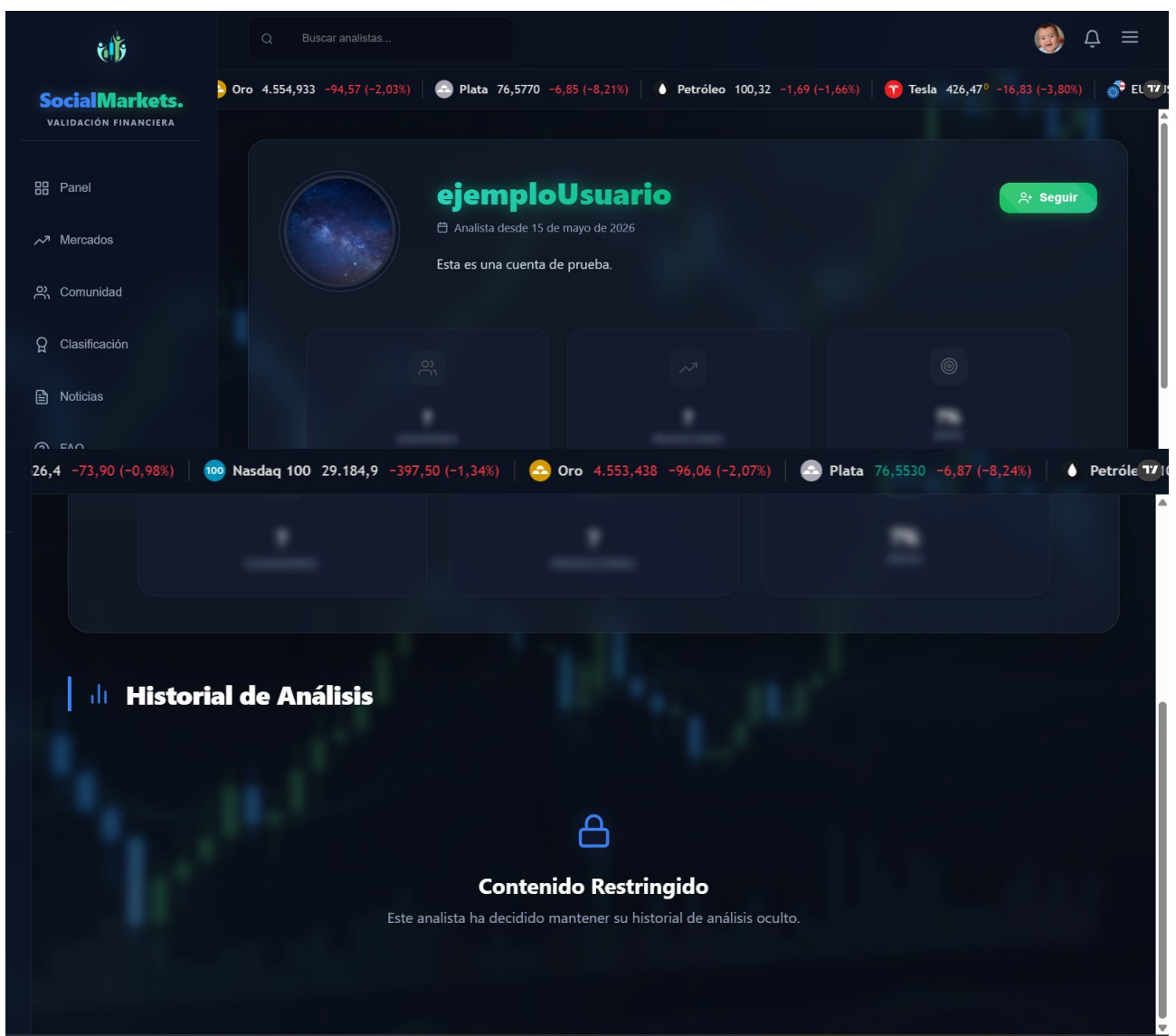
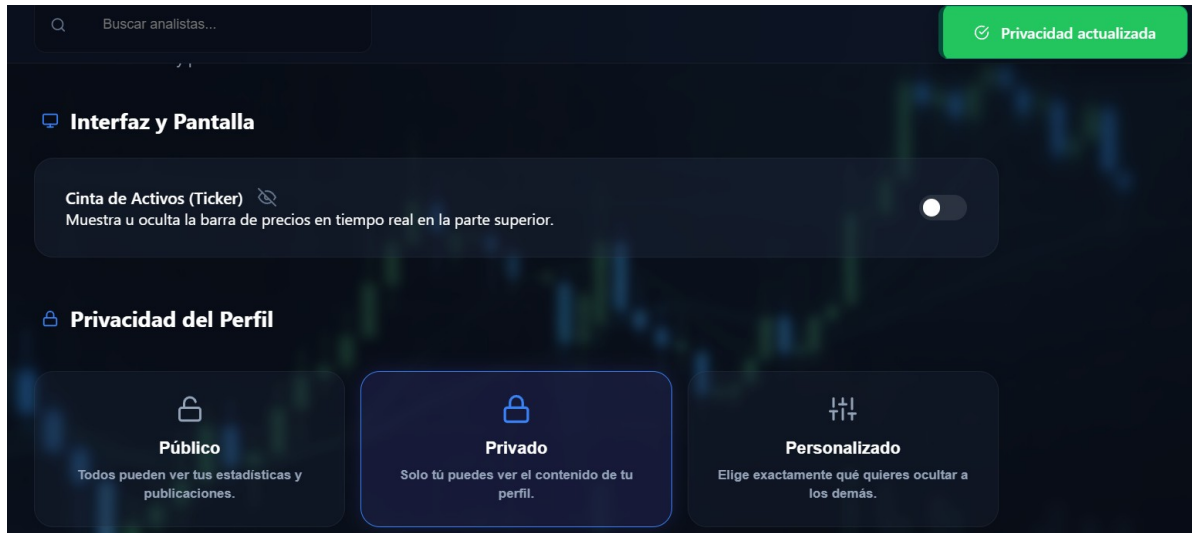




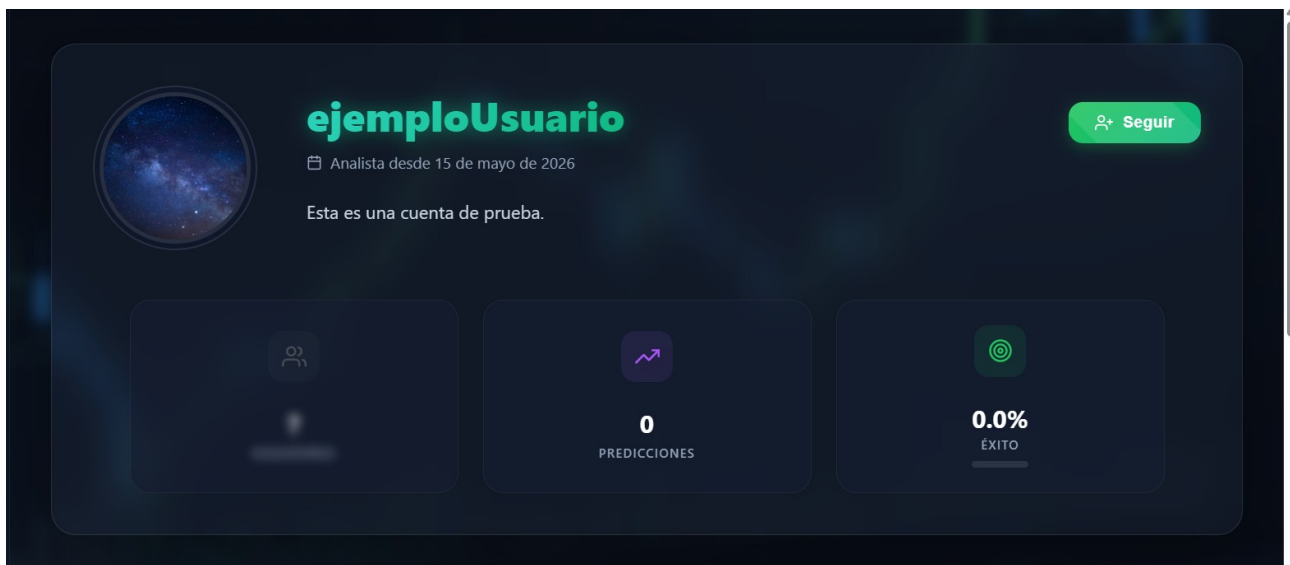
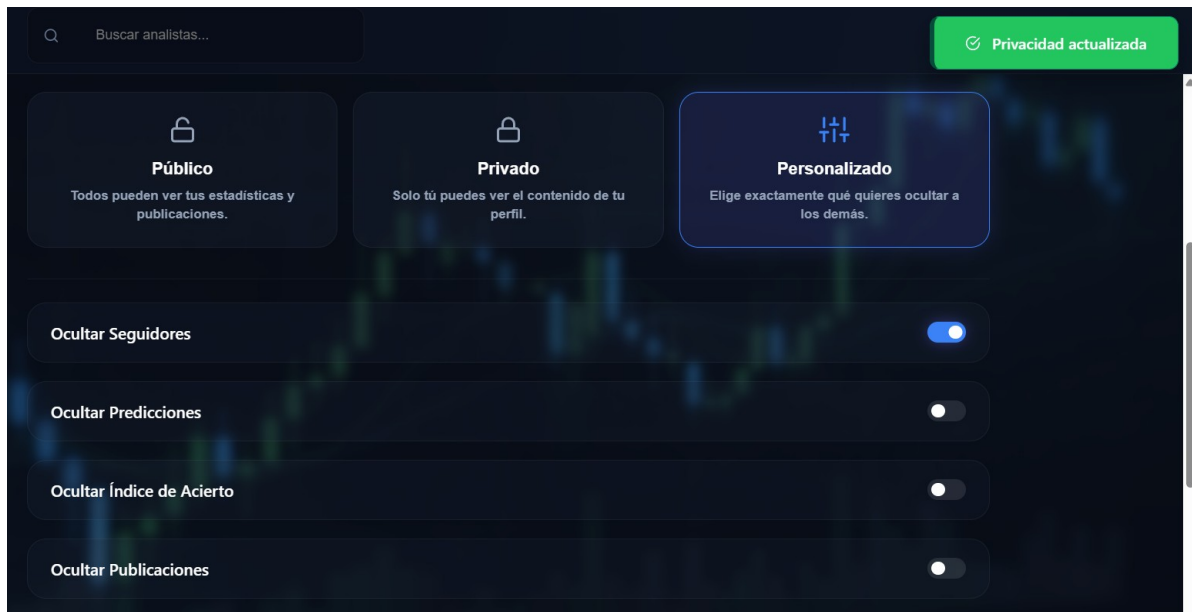
Al desactivar la Cinta de Activos, desaparecerá de la interfaz:



Si el usuario hace que su perfil sea privado, se ocultará su número de seguidores, número de predicciones, rendimiento y su historial al resto de usuario (en la segunda y tercera captura se puede ver como se vería el perfil del usuario privado desde otra cuenta):



Si el usuario únicamente hace privado únicamente el número de seguidores (o cualquier otra estadística), el resto se podrá ver sin problema (segunda captura):



El usuario también podrá cambiar su contraseña, cerrar sesión y eliminar su cuenta desde la configuración:

 **Cuenta**

 **Seguridad de Acceso**
Gestiona tu contraseña.

Contraseña Actual

Nueva Contraseña

Repetir Nueva Contraseña

 **GUARDAR**

Cancelar

Cerrar Sesión
Finaliza tu sesión actual.

 **Cerrar Sesión**

Borrar Cuenta
Elimina permanentemente tu cuenta y todos tus datos.

 **Borrar Cuenta**

Si el usuario busca el nombre usuario en la barra de búsqueda, saldrán los usuarios que más coincidan con su búsqueda (se mostrará su índice de acierto y número de predicciones para dar la máxima información posible):


SocialMarkets.
VALIDACIÓN FINANCIERA

Panel

Mercados

Comunidad

Clasificación

Noticias

FAQ



ejemplo



 **ejemploUsuario**
0.0% acierto 0 predicciones

 **ejemplo**
0.0% acierto 1 predicciones

 **ejemplo2**
0.0% acierto 0 predicciones

 **pruebaEjemploMensaje**
0.0% acierto 0 predicciones

0.0%

 PROYECCIONES TOTALES

0

Nivel: Analista Novato

 ANÁLISIS EN CURSO

0

0 Ganadoras 0 Perdedoras

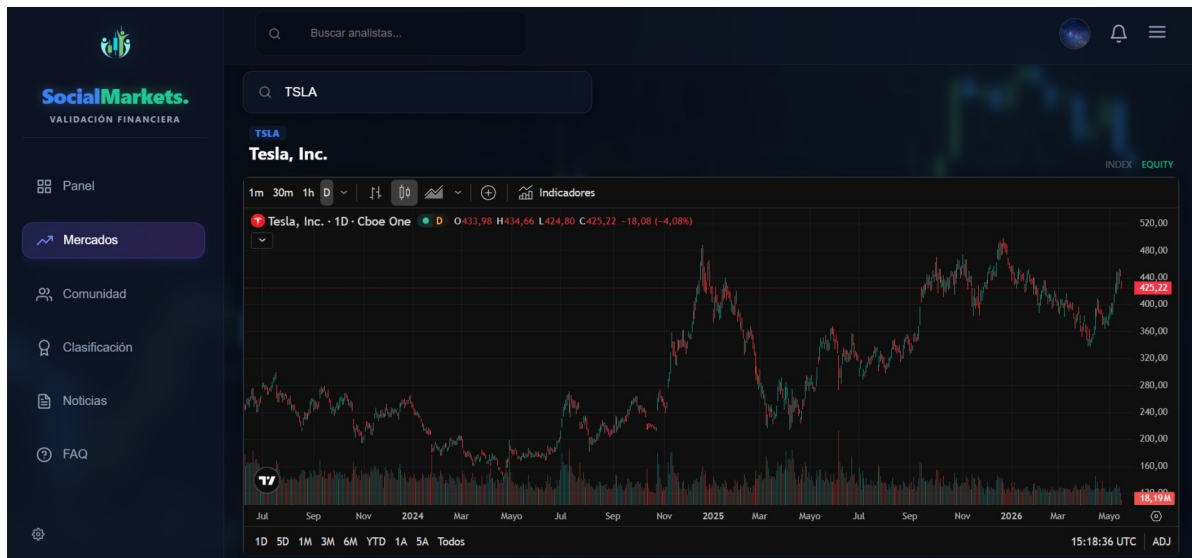
 **Actividad Reciente**

Todavía no has publicado ningún análisis.

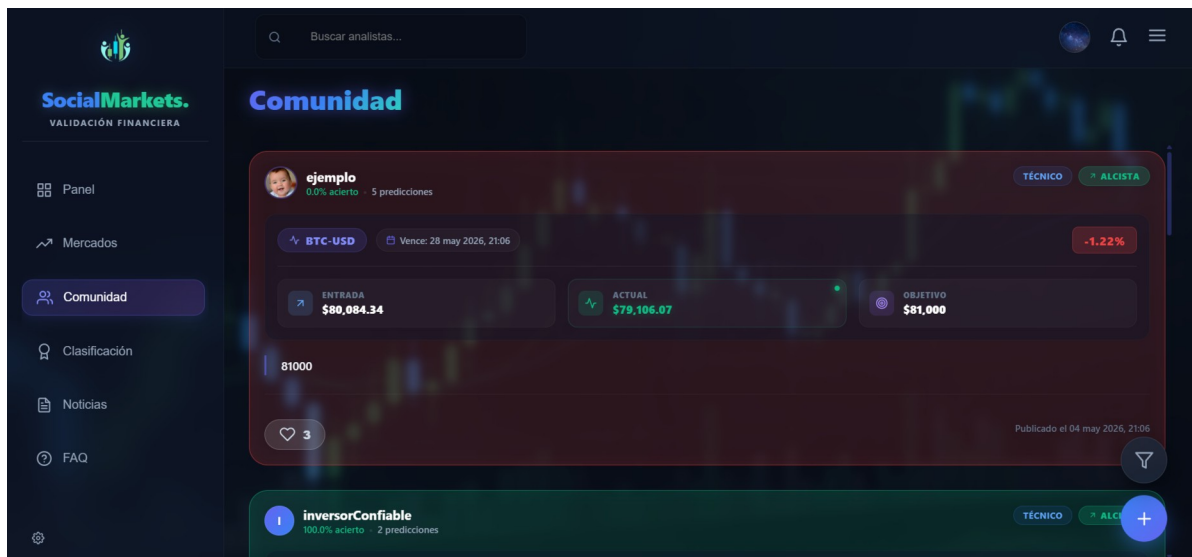
En el apartado de mercados, el usuario podrá buscar cualquier activo para poder ver el gráfico con su precio en tiempo real (gracias a la API de TradingView), y le saldrán todos los activos que más coincidan con lo que el usuario haya introducido:



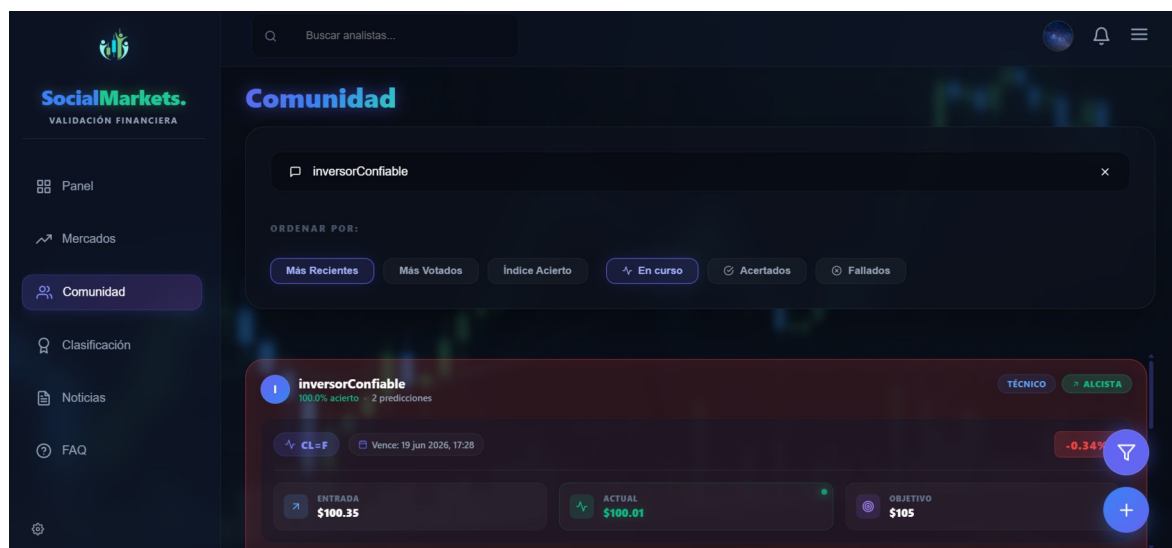
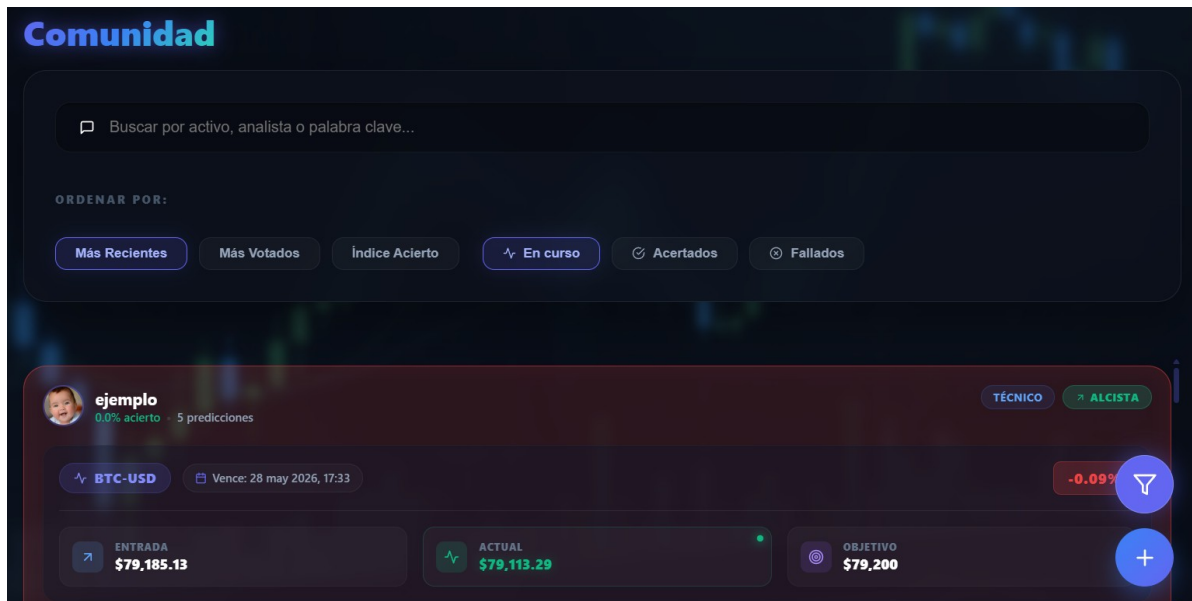
Al pulsar en el primero (la acción de la empresa Tesla), el usuario accederá al gráfico con su cotización actual:



La siguiente sección, la comunidad, es la más importante y la que más tiempo de desarrollo ha tenido. Cuando el usuario entra en la comunidad se podrán ver los análisis de los usuarios (por defecto ordenados por número de me gusta):

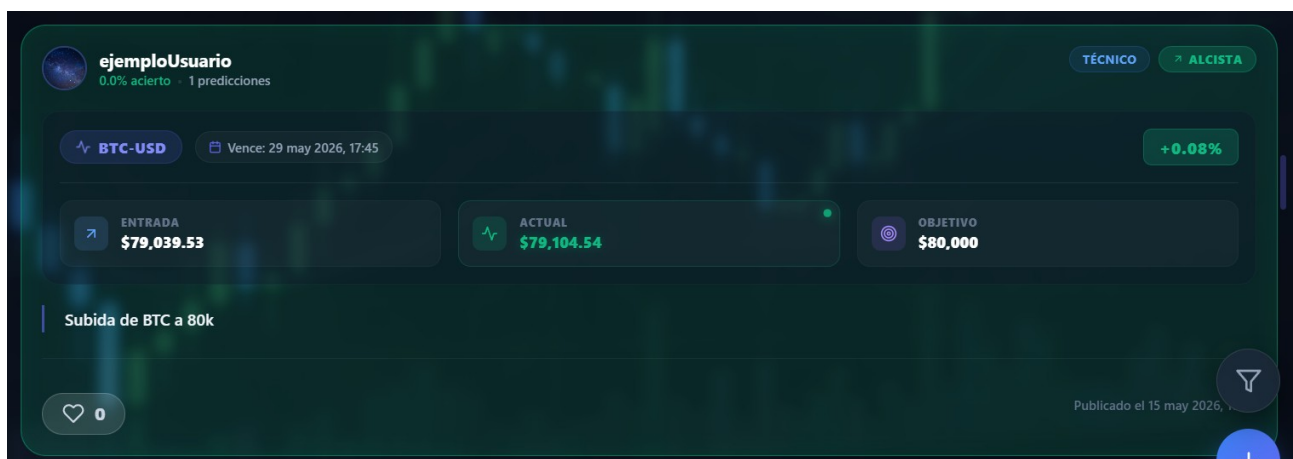
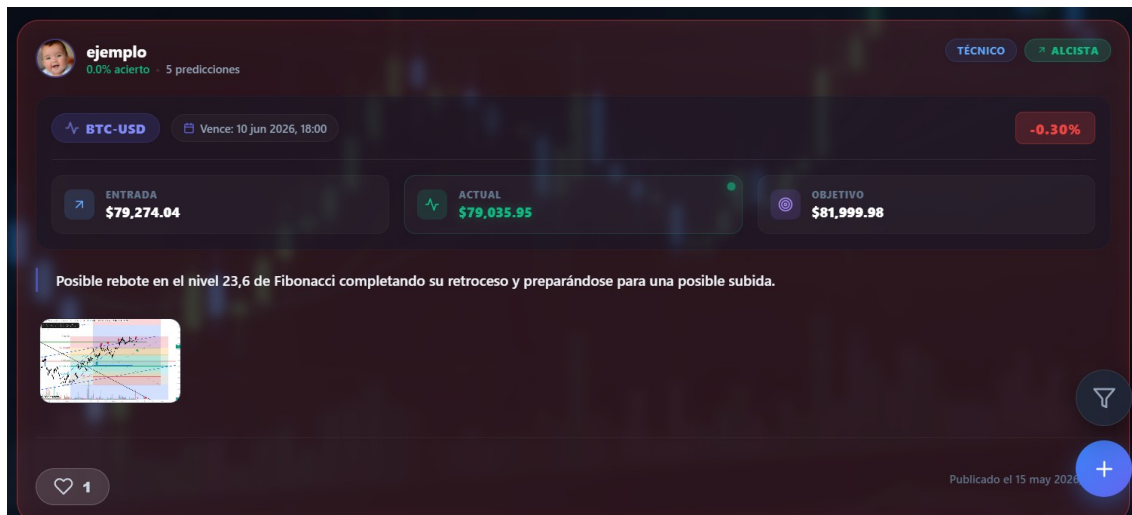


Si se pulsa en el icono del embudo abajo a la derecha, saldrá un panel con los filtros para poder ordenar los posts de la comunidad al gusto del usuario, o buscar análisis realizados por una persona en específico:

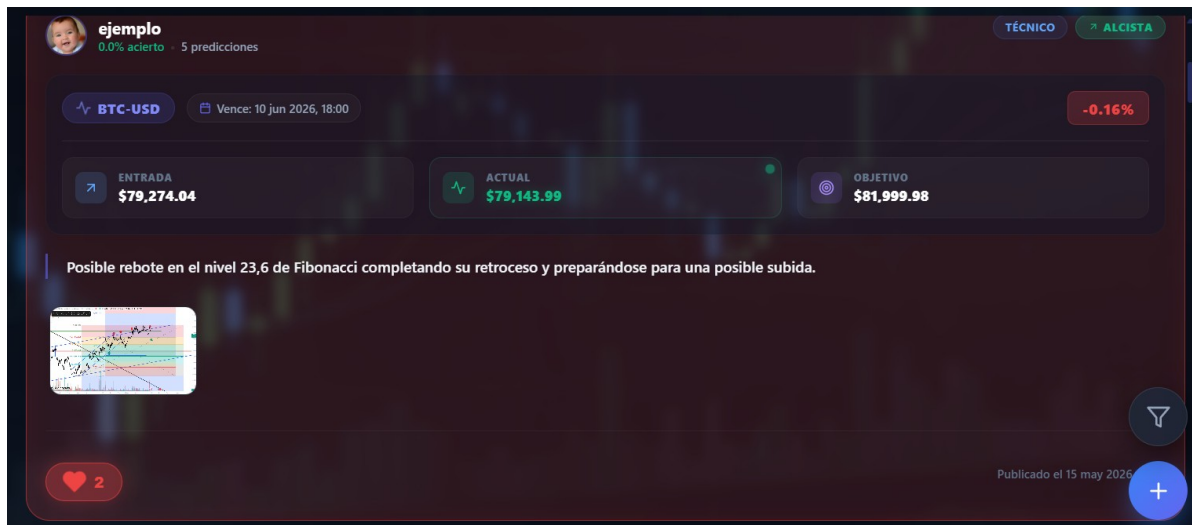


El color del fondo del análisis hace referencia a si el análisis se está cumpliendo en el momento actual o si al contrario no está siendo acertado. Si el fondo es rojo significará que el análisis actualmente es una posición en negativo, y si está en verde, será una posición positiva. En los análisis se podrá ver el precio de entrada al momento del análisis (el precio al que cotizaba el activo cuando el usuario publicó el análisis), la cotización actual del activo, y el precio objetivo que ha puesto el usuario como predicción según su análisis, además de otra información como el nombre del activo, la fecha de vencimiento (si al llegar a la fecha el precio de cotización del activo está por debajo del objetivo se dará como fallido, y si llega al precio objetivo antes de la fecha, se dará como acertado), el porcentaje de

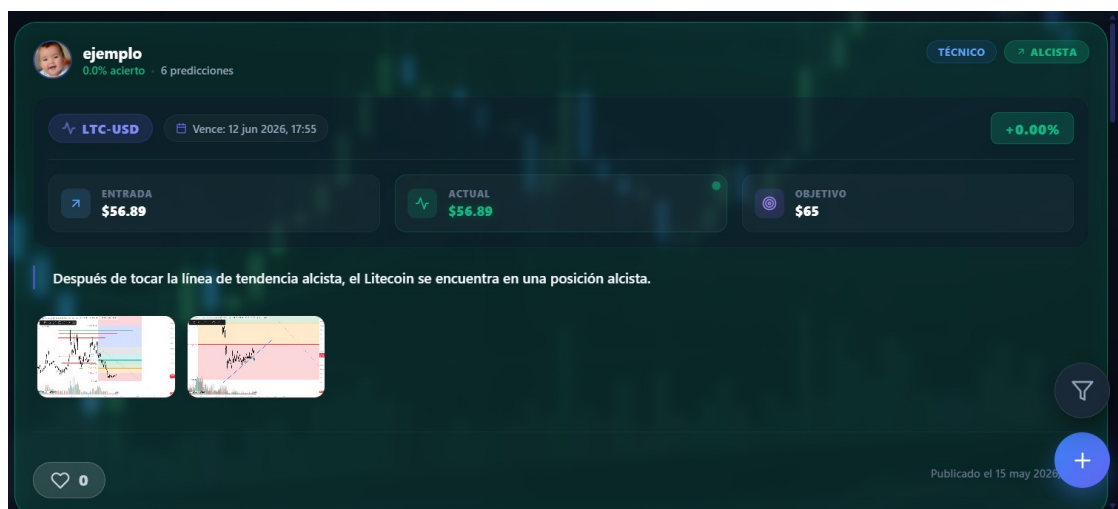
variación (lo que ha cambiado el activo según la posición de corto o largo desde el precio que ha determinado el usuario), la fecha de publicación, el tipo (técnico o fundamental) y el número de me gusta:



Al dar me gusta, el icono del corazón, abajo a la izquierda del análisis, cambiará de color y fondo:



Si el usuario ha publicado el análisis junto a una imagen, cualquier usuario podrá pulsarla para verla más de cerca, y si hay varias, podrá pasar entre ellas con unas flechas interactivas:





Al pulsar en el botón de la cruz abajo a la derecha (“+”), el usuario podrá publicar su propio análisis, y tendrá que rellenar el siguiente formulario:



Crear Análisis

Activo (Busca por símbolo o nombre)

Ej: BTC, Apple, Tesla...

Tipo de Análisis

Técnico

Fundamental

Precio Objetivo

\$ 0.00

Fecha de Vencimiento Estimada

mm/dd/yyyy --:-- --

Contenido del Análisis

Explica tu análisis...

0 / 1000

Imágenes (Máx. 4)



Subir

PUBLICAR ANÁLISIS



Al escribir un activo en la barra de búsqueda, saldrán los activos que más coincidan con lo que escribe el usuario:



Al seleccionar un activo (por ejemplo, “Apple Inc.”), saldrá el valor de cotización actual, con una actualización cada 10 segundos. De esta manera el usuario podrá saber el precio del activo antes de publicar cualquier análisis:



Al introducir el precio objetivo, el usuario podrá ver la diferencia de precio respecto al precio de cotización actual, y podrá establecer la fecha de vencimiento (si llega a

la fecha y el activo no llega al precio objetivo, se considerará un análisis fallido), si su análisis es técnico (fundamentado en patrones de gráficos y estructuras) o fundamental (fundamentado en noticias y emociones del mercado), una descripción obligatoria de por qué ha decidido marcar ese precio objetivo, e incluir una o varias imágenes (hasta 4) de forma opcional, que puedan ayudar a respaldar su análisis:

Crear Análisis

Activo (Busca por símbolo o nombre)
AAPL - Apple Inc.

PRECIO ACTUAL
301,23 US\$ ▲ 1.01% **EN VIVO**

Próxima actualización en 8s

Tipo de Análisis: **Técnico** Fundamental

Precio Objetivo: \$ 290
ANÁLISIS BAJISTA -3.73%

Fecha de Vencimiento Estimada
06/24/2026 06:06 PM

Técnico Fundamental \$ 290
ANÁLISIS BAJISTA -3.72%

Fecha de Vencimiento Estimada
06/24/2026 06:06 PM

Contenido del Análisis

Un agotamiento del volumen podría hacer que la oferta gane y el precio de la acción caiga al primer nivel de fibonacci, llegando a los 290\$.

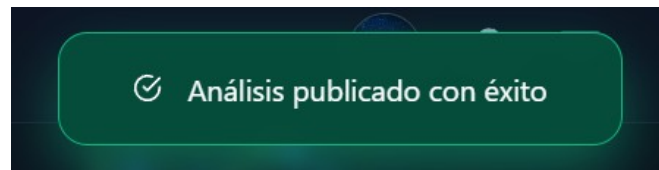
140 / 1000

Imágenes (Máx. 4)

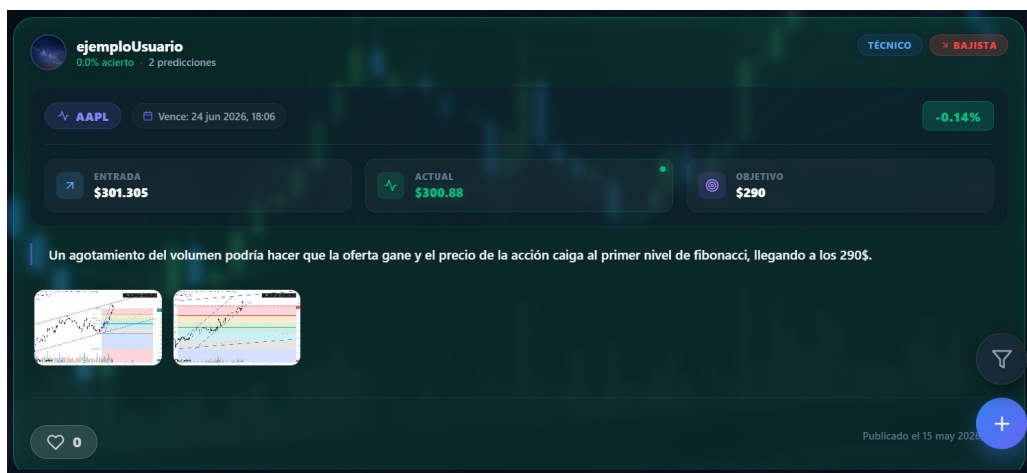
Subir

PUBLICAR ANÁLISIS

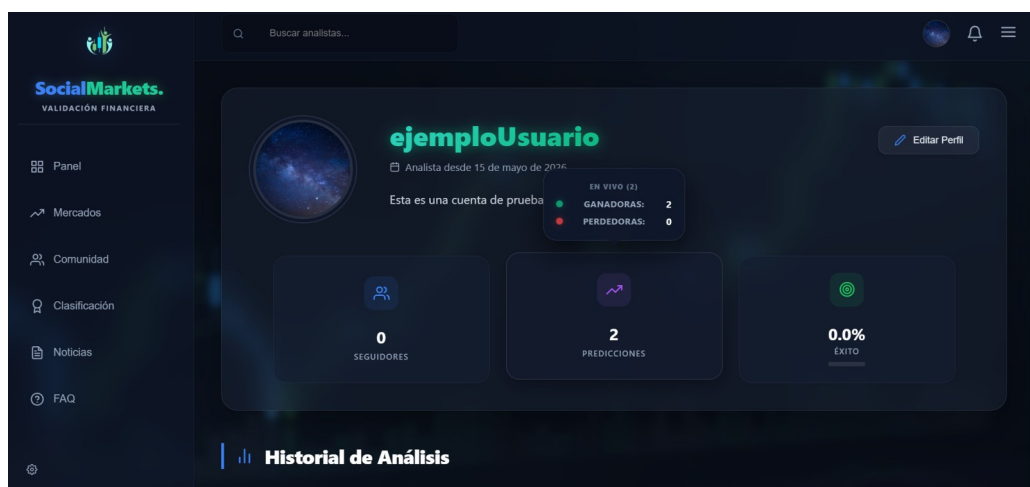
El usuario recibirá una notificación de que el análisis se ha publicado con éxito, en caso de no dar ningún fallo:



Análisis publicado (el fondo está en verde y marca -0,14%, ya que al ser una posición bajista el análisis está siendo acertado):



Ahora el usuario en su perfil podrá ver las estadísticas que tiene sobre sus análisis y su propio historial:



Historial de Análisis

 ejemploUsuario
0.0% acierto

TÉCNICO

BAJISTA

AAPL

Vence: 24 jun 2026, 18:06

-0.03%



ENTRADA

\$301.305



ACTUAL

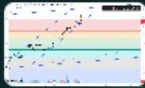
\$301.215



OBJETIVO

\$290

Un agotamiento del volumen podría hacer que la oferta gane y el precio de la acción caiga al primer nivel de fibonacci, llegando a los 290\$.



0

Publicado el 15 may 2026, 18:14

 ejemploUsuario
0.0% acierto

TÉCNICO

ALCISTA

BTC-USD

Vence: 29 may 2026, 17:45

+0.21%



ENTRADA

\$79,039.53



ACTUAL

\$79,205.51



OBJETIVO

\$80,000

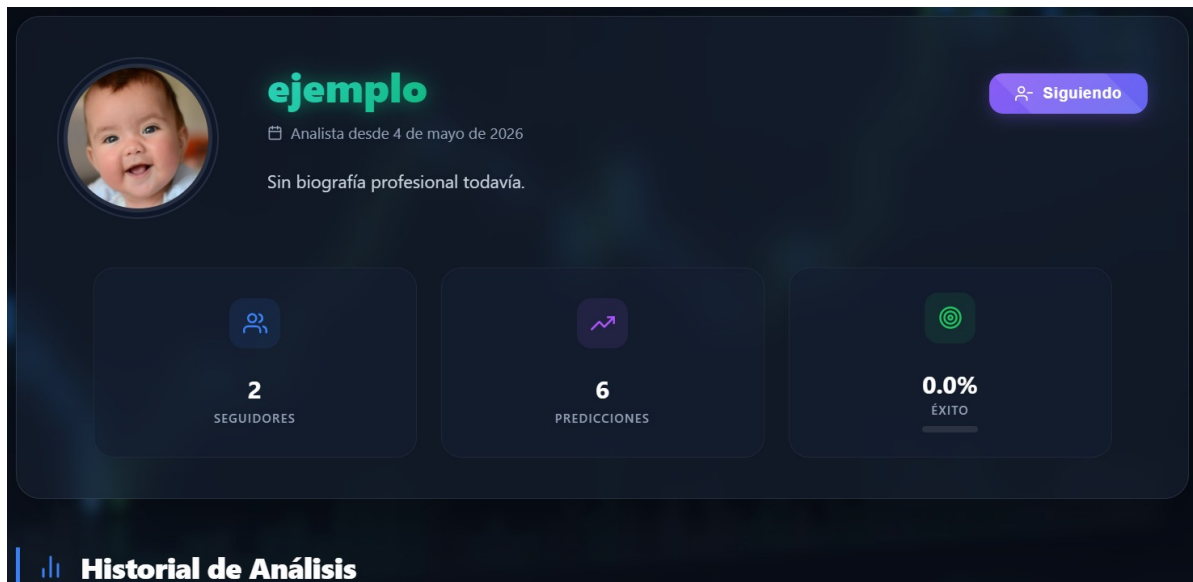
Subida de BTC a 80k



0

Publicado el 15 may 2026, 17:45

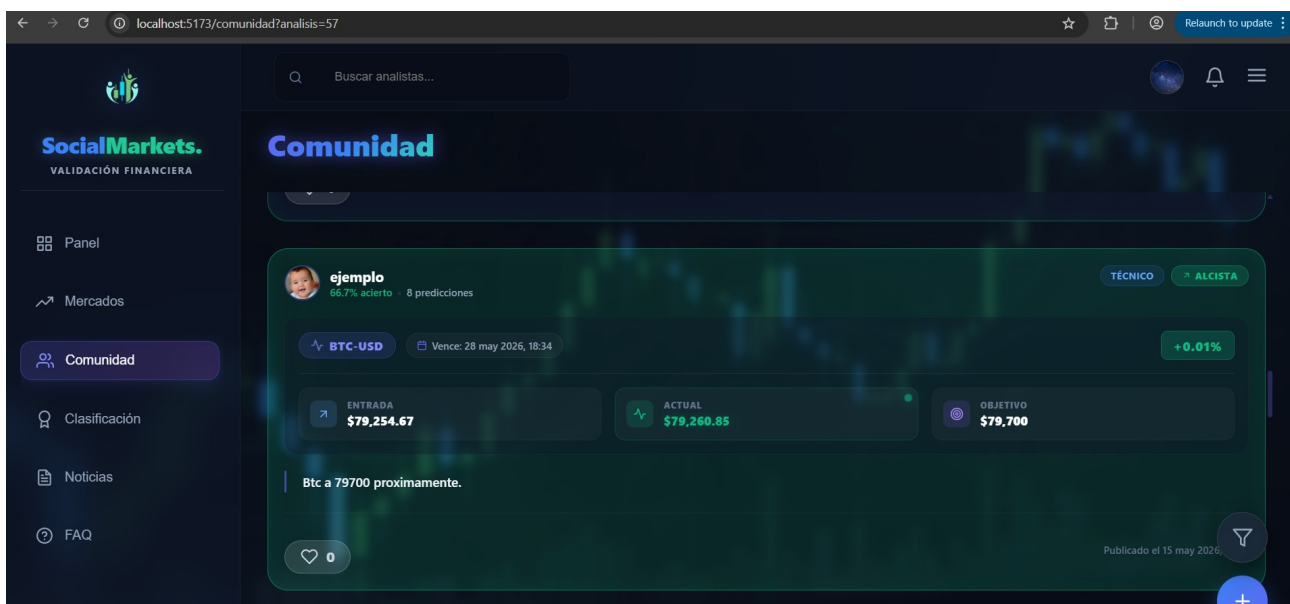
Poder ver el historial específico de cada usuario es muy útil al buscar el perfil de un usuario para poder ver qué predicciones ha realizado, y si nos gusta su manera de analizar, seguirle:



Esta interacción hará que el usuario seguido reciba una notificación nueva:



Si el usuario sigue a alguien y publica un análisis, le saldrá una notificación con ese aviso (al pulsar la notificación le llevará al análisis en concreto):



Si un usuario da me gusta a un análisis, también le llegará la notificación al autor de éste:



Todas las notificaciones podrán ser desactivadas a preferencia del usuario:



El siguiente apartado es el de la clasificación, con tres tipos (mejor índice de acierto, mayor número de predicciones, y más predicciones acertadas):

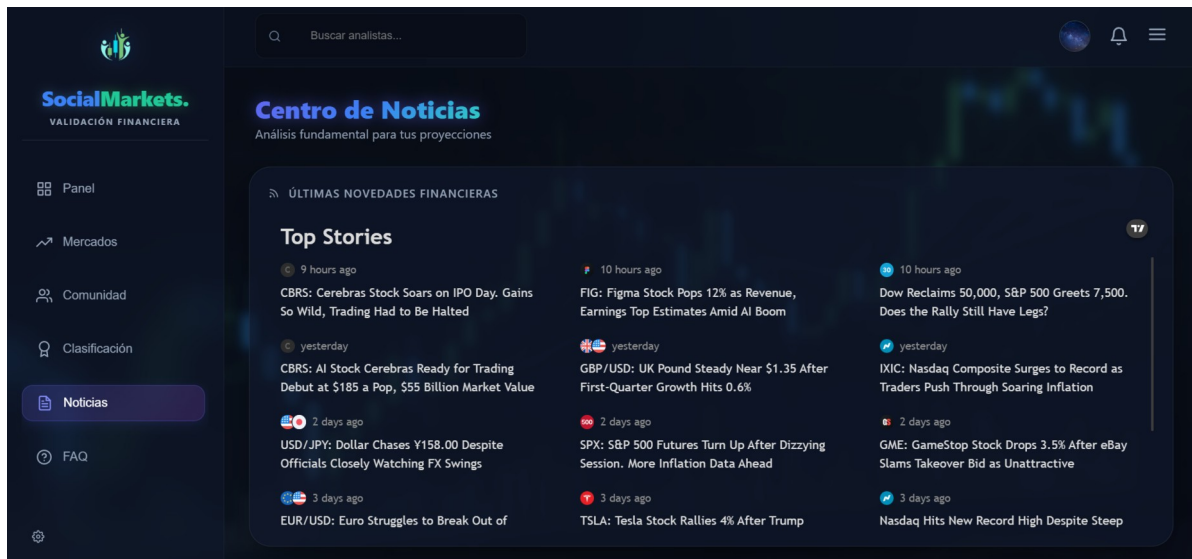




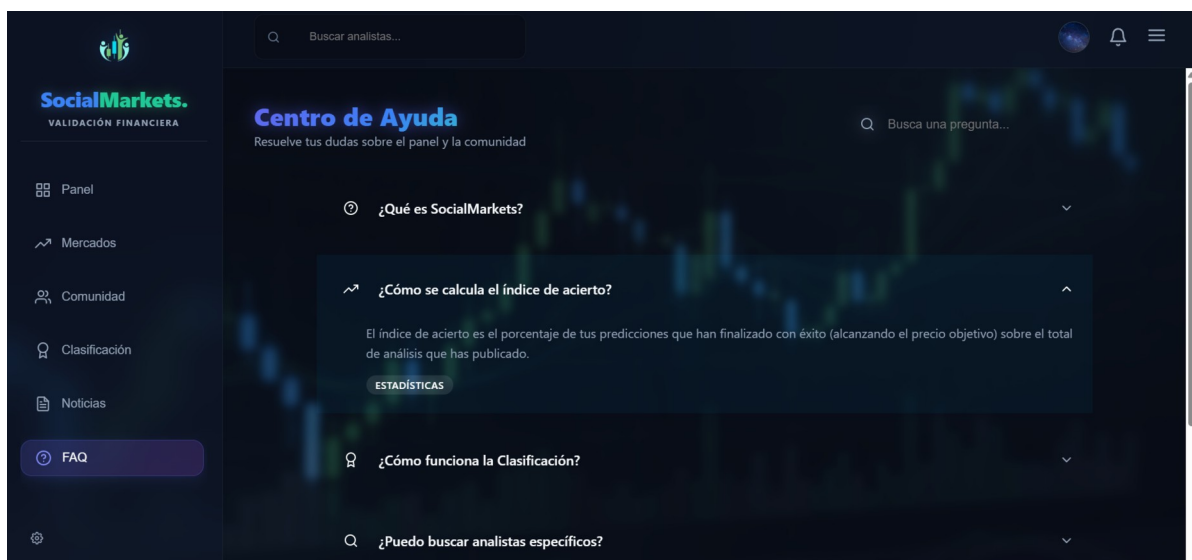
Todas las clasificaciones cuentan hasta con 10 puestos:

#4	SocialMarkets	0.0%	0 Predicciones	0 Acertadas
#5	lo	0.0%	0 Predicciones	0 Acertadas
#6	ejemplo2	0.0%	0 Predicciones	0 Acertadas
#7	prueba_usuario4	0.0%	0 Predicciones	0 Acertadas
#8	prueba_usuario1	0.0%	0 Predicciones	0 Acertadas
#9	prueba_usuario7	0.0%	0 Predicciones	0 Acertadas
#10	pruebaEjemploMensaje	0.0%	0 Predicciones	0 Acertadas

A continuación, el apartado de noticias, en el que el usuario podrá acceder a las noticias más recientes y de más relevancia (obtenidas a través de la API de TradingView):

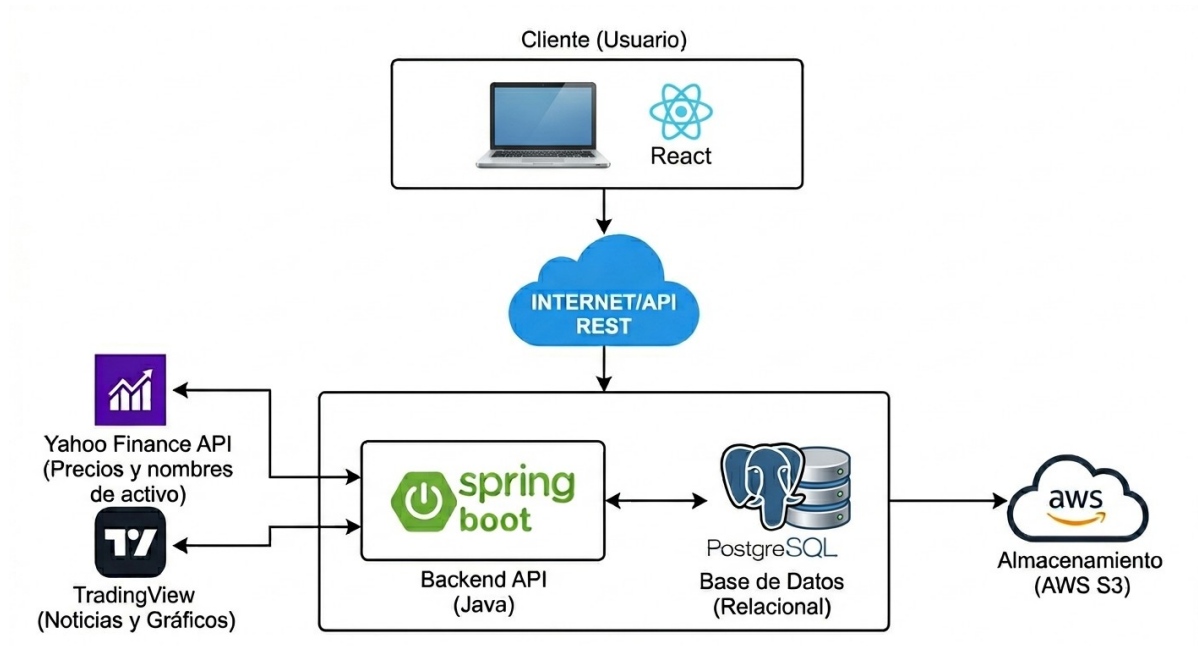


Y por último el apartado de preguntas frecuentes (FAQ), que servirá de ayuda al usuario a la hora de iniciarse en la aplicación o por si tiene alguna duda:



6.2. Arquitectura del sistema

El sistema utilizado en el proyecto tiene la siguiente arquitectura de software:



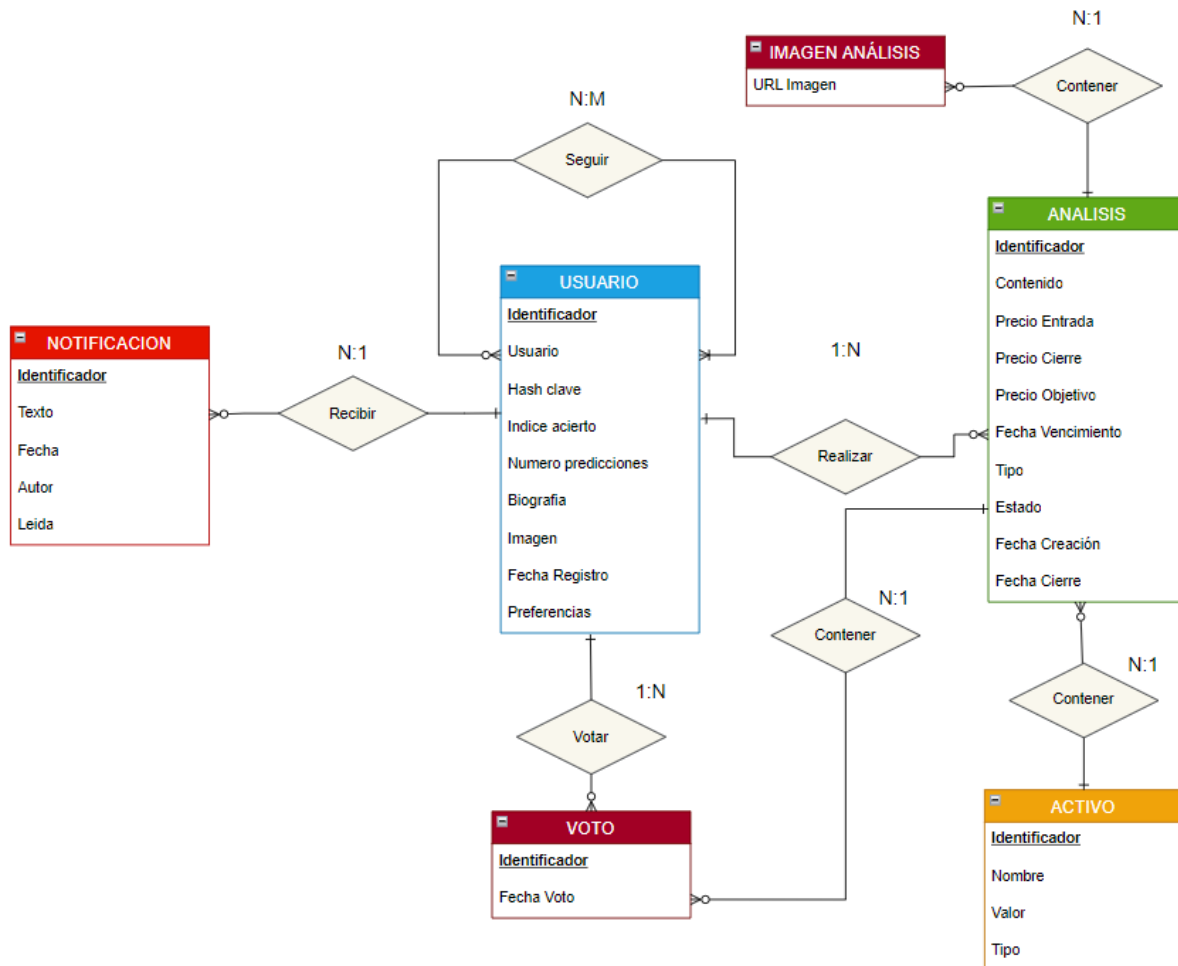
Como se puede observar en el diagrama de arquitectura, el sistema **SocialMarkets** se ha diseñado bajo un modelo de arquitectura limpia, permitiendo que cada componente cumpla un rol específico dentro del ecosistema financiero. La estructura se divide en los siguientes bloques principales:

- **Cliente (Frontend):** Representa la interfaz con la que interactúa el usuario final desde su navegador o dispositivo móvil. Está desarrollado íntegramente en React, lo que permite una navegación fluida y reactiva. Este componente es el encargado de consumir los servicios del backend y de renderizar las herramientas visuales pesadas, como los gráficos de TradingView, asegurando que la carga de procesamiento visual recaiga en el cliente para no saturar el servidor.
- **Capa de Comunicación (Internet / API REST):** Actúa como el puente entre el cliente y el servidor. Todas las peticiones se realizan de forma segura a través del protocolo HTTP, utilizando una API REST bien definida. En esta

capa se gestiona la seguridad mediante tokens JWT, garantizando que solo los usuarios autenticados puedan acceder a los recursos privados de la plataforma.

- **Servicios Externos (APIs Financieras):** Son los "motores de datos" que dan la inteligencia al sistema.
 - **Yahoo Finance API:** El servidor se conecta a esta fuente para obtener cotizaciones reales y nombres de activos, permitiendo que se validen las predicciones de forma objetiva.
 - **TradingView:** El cliente web integra directamente sus servicios para ofrecer una terminal técnica profesional con noticias y gráficos avanzados sin necesidad de desarrollar un motor gráfico propio desde cero.
- **Entorno del Servidor (Backend):** Es el corazón de la aplicación, donde reside la lógica de negocio.
 - **Backend API (Spring Boot):** Desarrollado en Java, este componente gestiona desde el registro de usuarios hasta el cálculo de los índices de acierto. Se encarga de coordinar la comunicación con la base de datos y los servicios externos.
 - **Base de Datos (PostgreSQL):** Se utiliza un sistema gestor relacional para almacenar toda la información estructural del proyecto (usuarios, seguidores, análisis, métricas de ranking, etc.), garantizando la integridad referencial y la persistencia de los datos.
- **Almacenamiento en la Nube (AWS S3):** Para evitar la sobrecarga del servidor y mejorar la velocidad de carga, las imágenes de perfil y archivos multimedia se alojan de forma independiente en Amazon S3. Esto permite que el sistema sea más escalable y que la entrega de contenido sea mucho más eficiente.

6.3. Diagrama entidad/relación



A continuación, se describen las entidades que conforman el núcleo del sistema, detallando sus atributos principales y las relaciones que permiten el funcionamiento de la plataforma social y de análisis financiero:

1. Usuario

Esta entidad representa a los analistas y miembros de la comunidad. Almacena la información de perfil y las métricas de rendimiento necesarias para el motor de ranking de la aplicación. Cuenta con un identificador único (clave primaria), nombre de usuario, contraseña cifrada (hash), biografía, imagen de perfil y fecha de registro. Además, incluye campos técnicos como el índice de acierto y el número de predicciones para medir la reputación del analista, así como un campo de preferencias para la configuración de privacidad y notificaciones. Esta entidad presenta una relación reflexiva de cardinalidad N:M (Seguir) para gestionar el sistema de seguidores, y relaciones 1:N con las entidades Análisis, Voto y Notificación.

2. Análisis

Es la entidad central del sistema, donde se registra la actividad predictiva sobre los mercados. Contiene el identificador único, el contenido textual de la tesis de inversión, y los niveles de precio clave: entrada, cierre y objetivo. Incluye datos temporales como la fecha de creación, vencimiento y cierre, además del tipo de análisis y su estado actual (abierto, acertado o fallido). Se relaciona de forma N:1 con Usuario (un analista puede realizar múltiples análisis) y con Activo (un análisis se centra en un único instrumento financiero). Asimismo, posee una relación 1:N con Imagen Análisis para que se puedan adjuntar gráficos técnicos o noticias.

3. Voto

Actúa como la entidad de interacción social entre los usuarios y los análisis publicados. Registra la fecha en la que se produce la interacción y sirve de referencia para valorar el apoyo de la comunidad a una tesis de inversión. Se relaciona con cardinalidad N:1 tanto con Usuario como con Análisis, garantizando que cada registro de voto pertenezca a un único autor y a una publicación específica.

4. Activo

Esta entidad almacena los instrumentos financieros (acciones, criptomonedas, divisas, etc.) sobre los que se puede operar. Contiene un identificador, el nombre del activo, su valor actual de mercado y el tipo de activo. Mantiene una relación 1:N con Análisis, permitiendo que un mismo activo financiero sea objeto de estudio en múltiples publicaciones de diferentes analistas.

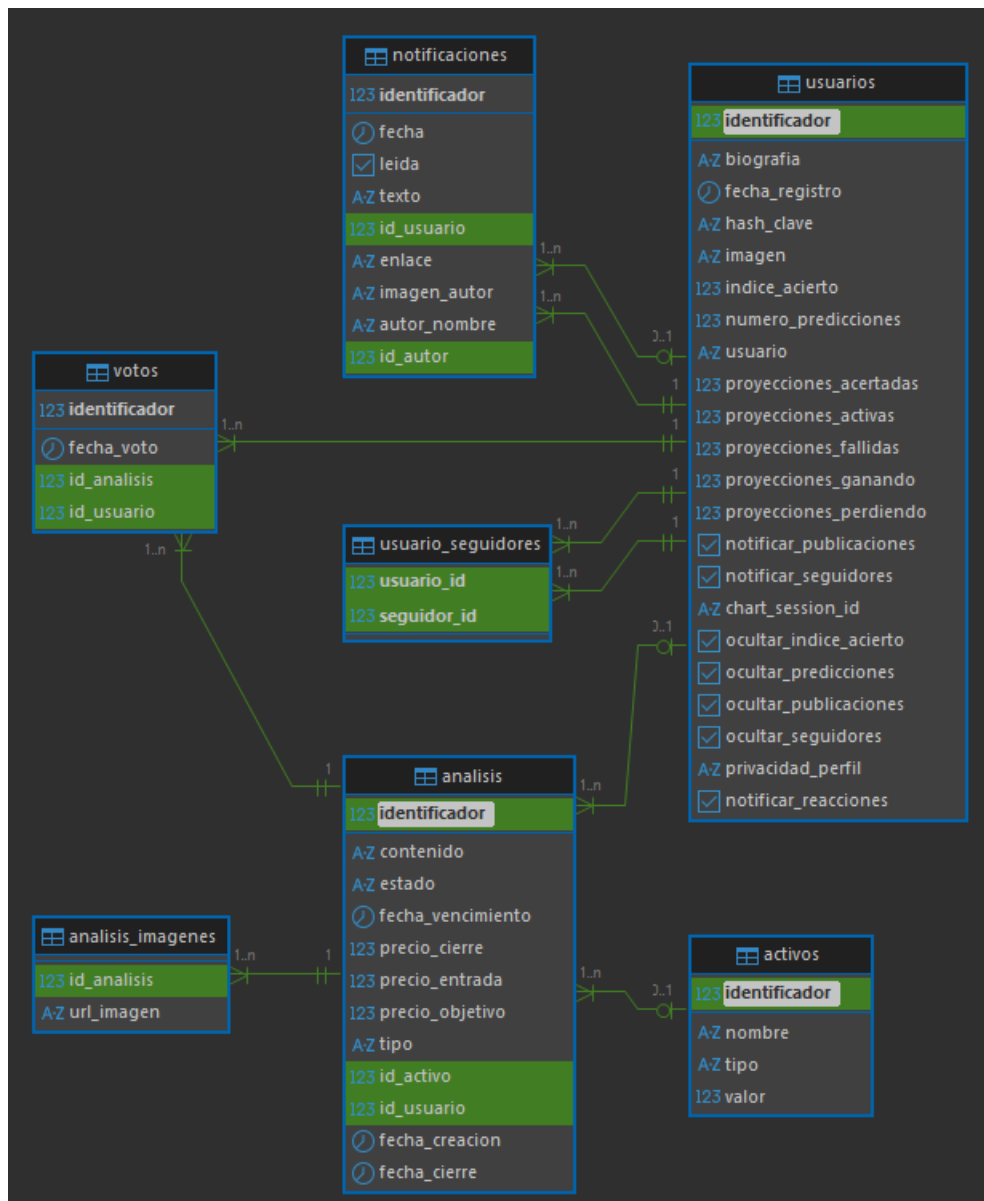
5. Notificación

Gestiona el sistema de alertas de la plataforma. Almacena el identificador, el texto del mensaje, la fecha de emisión, el autor que desencadenó la alerta y un estado booleano para controlar si el aviso ha sido leído. Se une mediante una relación N:1 con Usuario, de modo que cada notificación está dirigida a un único destinatario.

6. Imagen Análisis

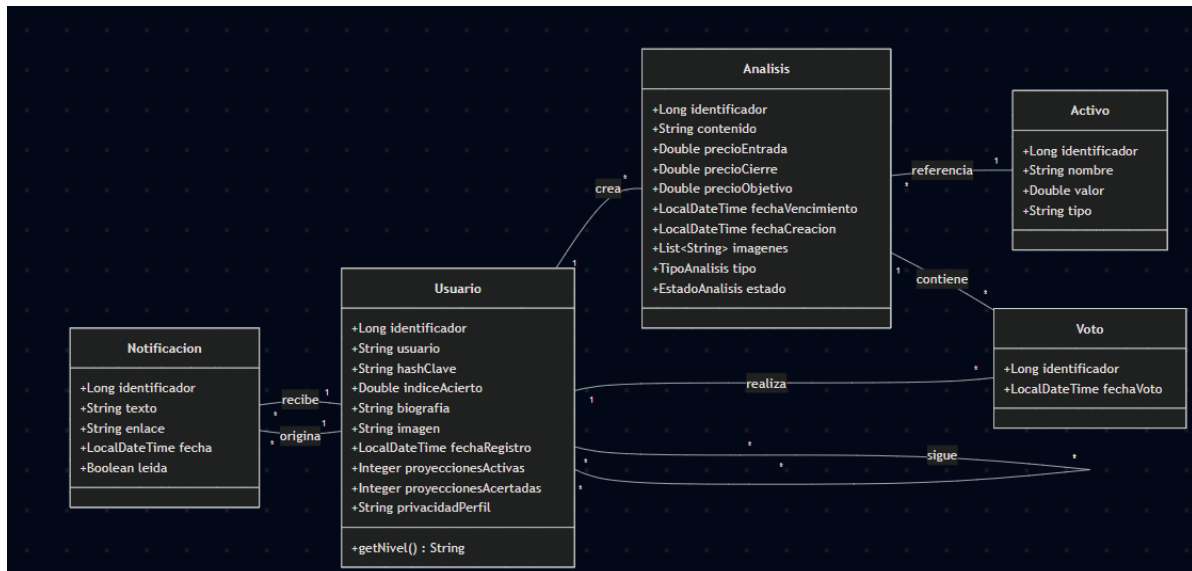
Funciona como una entidad de apoyo para el contenido visual de la plataforma. Almacena la URL o ruta de acceso a las capturas de pantalla de los gráficos financieros asociados a una tesis. Se relaciona con cardinalidad N:1 con Análisis, permitiendo que una publicación pueda contener diversas referencias visuales para respaldar la estrategia planteada.

6.4. Diagrama Relacional

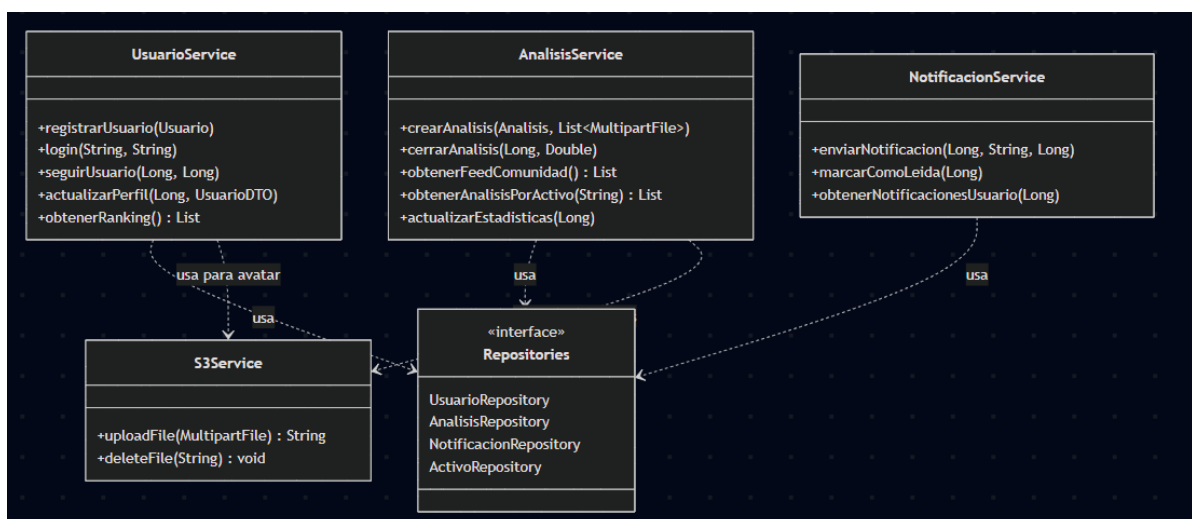


6.5. Diagrama de clases

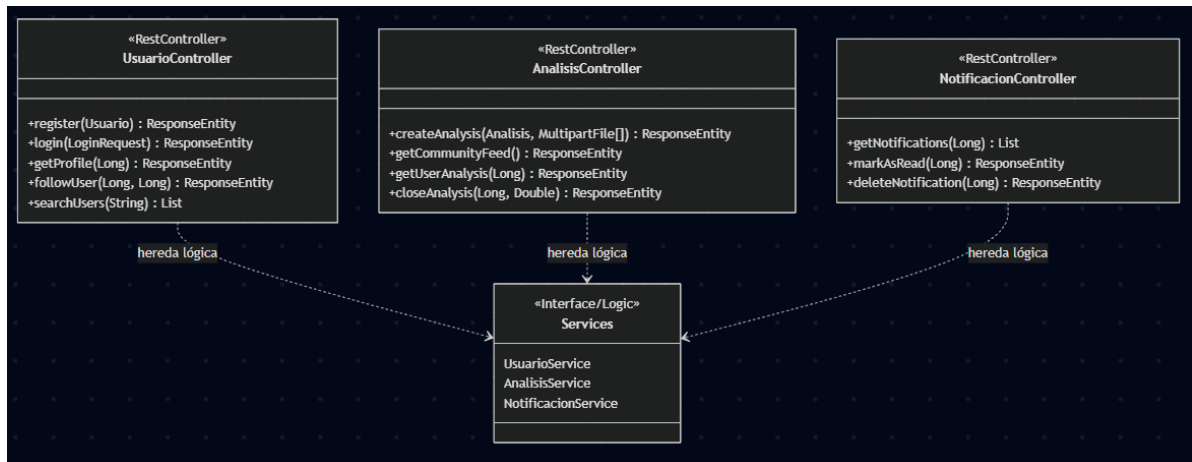
Entidades:



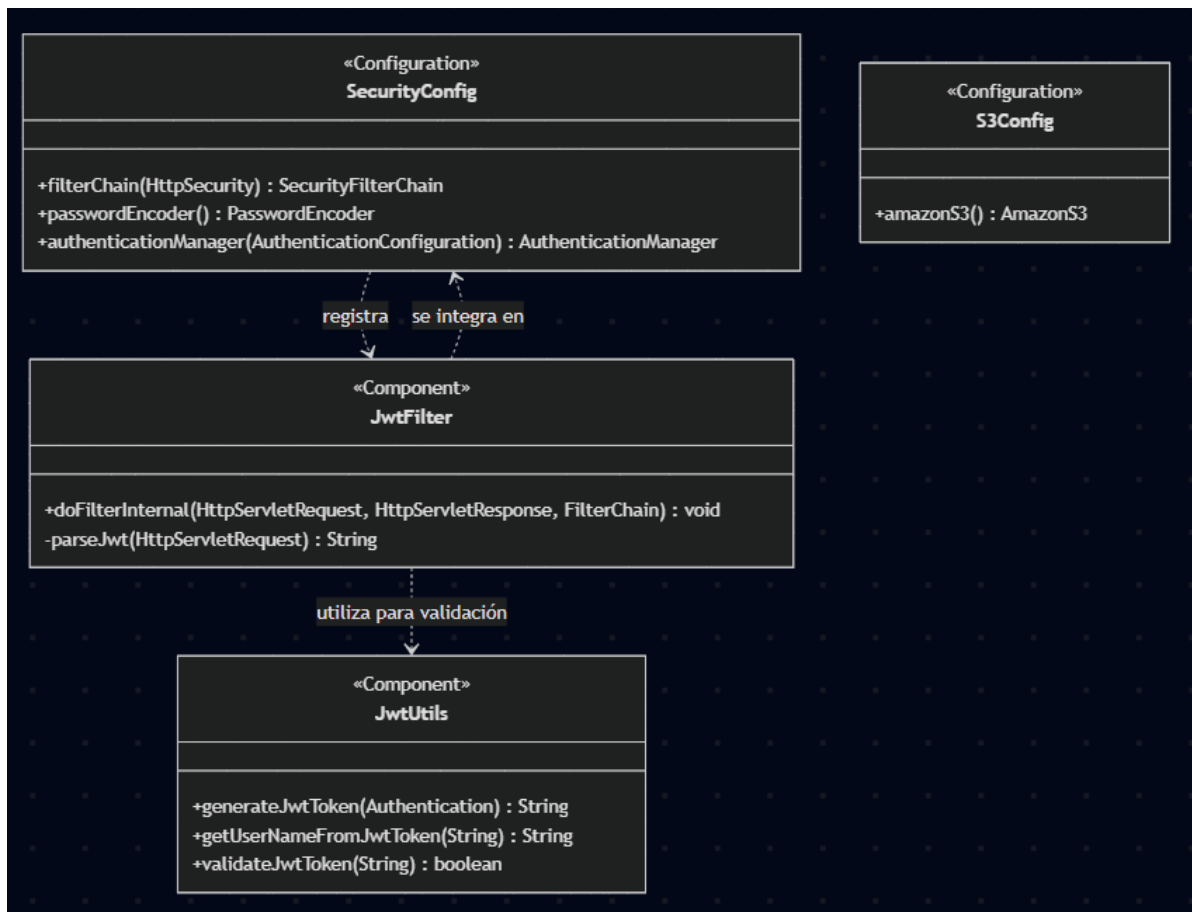
Servicios:



Controladores:



Seguridad y configuración:



7. Desarrollo de la aplicación

7.1. Tecnologías y herramientas utilizadas

En el desarrollo de **SocialMarkets** se ha seleccionado un stack tecnológico moderno y profesional, priorizando la escalabilidad del backend y la interactividad del frontend. A continuación, se detallan las herramientas utilizadas:

Lenguajes y Frameworks de Backend

- **Java 25:** Es el lenguaje principal del servidor. Se ha elegido por su robustez, su fuerte tipado y su excelente rendimiento en aplicaciones empresariales. Al ser el lenguaje con el que más se ha trabajado durante el grado, permite una implementación de código efectiva.
- **Spring Boot 4.0.5:** Framework líder para la creación de aplicaciones Java. Se ha utilizado para desarrollar la API REST debido a su capacidad para autoconfigurar servicios, su gestión eficiente de dependencias y su integración nativa con sistemas de seguridad y bases de datos.
- **Spring Security & JWT:** Se ha implementado para gestionar la autenticación y autorización. El uso de JSON Web Tokens (JWT) permite una seguridad ideal para APIs que conectan con clientes modernos como React.
- **Lombok:** Librería esencial para reducir el código redundante en Java. Mediante anotaciones, automatiza la creación de constructores, getters y setters, manteniendo las entidades y servicios mucho más limpios y legibles.

Lenguajes y Herramientas de Frontend

- **React 19:** Biblioteca de JavaScript utilizada para construir la interfaz de usuario. Su arquitectura basada en componentes y su capacidad para manejar estados de forma reactiva han sido fundamentales para crear una experiencia de "Terminal Social" fluida y dinámica.

- **Vite 8.0:** Se ha utilizado como entorno de desarrollo y empaquetador para el frontend. Destaca por su velocidad de compilación casi instantánea y su eficiencia al generar el paquete final de producción.
- **Vanilla CSS & Framer Motion:** Para el diseño visual se ha optado por CSS puro para tener un control total sobre el estilo de Glassmorphism. Framer Motion se ha integrado para añadir micro-animaciones y transiciones suaves que elevan la calidad percibida de la aplicación.
- **Axios:** Cliente HTTP utilizado en el frontend para realizar peticiones asíncronas a la API del backend, gestionando de forma sencilla los interceptores para el envío automático del token JWT.

Almacenamiento y Gestión de Datos

- **PostgreSQL 16:** Sistema gestor de base de datos relacional elegido para la persistencia principal. Su fiabilidad y soporte para consultas complejas aseguran la integridad de las relaciones entre usuarios, seguidores y análisis financieros.
- **Amazon Web Services (AWS S3):** Servicio de almacenamiento en la nube utilizado para hospedar de forma segura las imágenes de perfil y archivos multimedia. Esto permite que el servidor de aplicaciones sea más ligero y escalable al no tener que gestionar archivos en el sistema de ficheros local.

APIs e Integraciones Externas

- **Yahoo Finance API:** Fuente de datos principal para el backend. Permite obtener cotizaciones reales y precisas para alimentar el motor de cálculo del ranking de éxito de los analistas.
- **TradingView API:** Utilizada en el frontend para integrar gráficos técnicos profesionales y herramientas de dibujo, ofreciendo al usuario una experiencia de análisis de nivel institucional.

Entornos y Control de Software

- **VS Code & IntelliJ IDEA:** Se han utilizado estos IDEs especializados para el desarrollo de frontend y backend respectivamente, aprovechando sus herramientas de depuración y extensiones de productividad.

- **Git & GitHub:** Herramientas fundamentales para el control de versiones, permitiendo una gestión organizada del código, la trazabilidad de los cambios y el respaldo seguro del proyecto en la nube.

7.2. Descripción de las principales funcionalidades

1. Obtención de precios en tiempo real (Yahoo Finance API)

El motor del sistema depende de la capacidad de obtener precios precisos. En el backend, se ha implementado un servicio que consulta directamente a la API de Yahoo Finance, utilizando un sistema de caché temporal para optimizar el rendimiento y evitar bloqueos por exceso de peticiones.

```
// Obtiene el precio de mercado actual para un símbolo
public Double obtenerPrecioActual(String symbol) {
    long now = System.currentTimeMillis();

    // Si tenemos el precio en caché y no ha caducado, lo devolvemos directamente
    if (priceCache.containsKey(symbol) && (now - cacheTimestamps.get(symbol)) < CACHE_DURATION) {
        return priceCache.get(symbol);
    }

    try {
        // Consultamos a Yahoo Finance simulando un navegador para evitar bloqueos
        String url = "https://query2.finance.yahoo.com/v8/finance/chart/" + symbol + "?interval=1m&range=1d";
        HttpHeaders headers = new HttpHeaders();
        headers.set("User-Agent", "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/121.0.0.0");
        HttpEntity<String> entity = new HttpEntity<>(headers);
        ResponseEntity<String> response = restTemplate.exchange(url, HttpMethod.GET, entity, String.class);

        // Parseamos el JSON para extraer el precio actual
        ObjectMapper mapper = new ObjectMapper();
        JsonNode root = mapper.readTree(response.getBody());
        JsonNode meta = root.path("chart").path("result").get(0).path("meta");
        Double price = meta.path("regularMarketPrice").asDouble();

        // Guardamos en caché y devolvemos
        priceCache.put(symbol, price);
        cacheTimestamps.put(symbol, now);
        return price;
    } catch (Exception e) {
        System.err.println("Error obteniendo precio para " + symbol + ": " + e.getMessage());
        return null;
    }
}
```

2. Algoritmo de cálculo del Ranking de Analistas

Esta es la funcionalidad diferencial del proyecto. Se ha desarrollado una lógica que evalúa automáticamente el éxito de un analista basándose en sus predicciones. El sistema calcula la tasa de acierto comparando el precio objetivo con el precio

actual de mercado, actualizando el perfil del usuario de forma dinámica tras cada cierre de operación.

```
// Comprueba los análisis PENDIENTES contra los precios reales para ver si han tenido éxito o han fallado
@Transactional
public void verificarAnalisisPendientes() {
    try {
        List<Analisis> pendientes = analisisRepository.findByEstado(EstadoAnalisis.PENDIENTE);
        LocalDateTime ahora = LocalDateTime.now();

        for (Analisis a : pendientes) {
            try {
                if (a.getActivo() == null || a.getFechaVencimiento() == null) continue;

                boolean cerrado = false;
                Double precioActual = marketDataService.obtenerPrecioActual(a.getActivo().getNombre());

                if (precioActual != null) {
                    boolean isBullish = a.getPrecioObjetivo() > a.getPrecioEntrada();

                    // Si se toca el precio objetivo, marcamos como ACERTADO
                    if ((isBullish && precioActual >= a.getPrecioObjetivo()) ||
                        (!isBullish && precioActual <= a.getPrecioObjetivo())) {
                        a.setEstado(EstadoAnalisis.ACERTADO);
                        a.setPrecioCierre(precioActual);
                        a.setFechaCierre(ahora);
                        cerrado = true;
                    }

                    // Si llega la fecha de vencimiento sin tocar el objetivo, marcamos como FALLIDO
                    if (!cerrado && a.getFechaVencimiento().isBefore(ahora)) {
                        a.setEstado(EstadoAnalisis.FALLIDO);
                        a.setPrecioCierre(precioActual != null ? precioActual : a.getPrecioEntrada());
                        a.setFechaCierre(ahora);
                        cerrado = true;
                    }
                }

                if (cerrado) {
                    analisisRepository.save(a);
                    actualizarEstadisticasUsuario(a.getUsuario());
                }
            } catch (Exception e) {
                System.err.println("Error procesando análisis " + a.getIdentificador() + ": " + e.getMessage());
            }
        }
    } catch (Exception e) {
        System.err.println("Error general en verificación de análisis: " + e.getMessage());
    }
}
```

3. Integración de almacenamiento en la nube (AWS S3)

Para la gestión de imágenes de perfil y archivos multimedia, se ha integrado el servicio de Amazon Web Services (S3). Esto permite delegar la persistencia de

archivos a una infraestructura externa escalable, evitando ocupar espacio en el servidor de aplicaciones y mejorando significativamente la velocidad de entrega del contenido multimedia.

```
/**
 * Servicio para gestionar la subida de archivos (imágenes) a la nube de Amazon (AWS S3)
 */
@Service
public class S3Service {

    @Autowired
    private S3Client s3Client;

    @Value("${aws.s3.bucket-name}")
    private String bucketName;

    @Value("${aws.s3.region}")
    private String region;

    // Sube un archivo a S3 y devuelve la URL pública para poder visualizarlo
    public String subirArchivo(MultipartFile archivo) throws IOException {
        // Generamos un nombre único para el archivo para evitar que se sobrescriban
        String nombreArchivo = UUID.randomUUID().toString() + "_" + archivo.getOriginalFilename();

        // Configuramos la petición de subida
        PutObjectRequest putObjectRequest = PutObjectRequest.builder()
            .bucket(bucketName)
            .key(nombreArchivo)
            .contentType(archivo.getContentType())
            .build();

        // Enviamos el archivo a AWS
        s3Client.putObject(putObjectRequest, RequestBody.fromInputStream(archivo.getInputStream(), archivo.getSize()));

        // Devolvemos la dirección web (URL) del archivo recién subido
        return String.format("https://%s.s3.%s.amazonaws.com/%s", bucketName, region, nombreArchivo);
    }
}
```

4. Seguridad y protección de rutas con JWT

Para garantizar la seguridad de los datos, se ha implementado un sistema de autenticación basado en JSON Web Tokens (JWT). Cada petición del cliente es interceptada por un filtro de seguridad en el backend que valida la integridad del token, asegurando que solo los analistas autorizados puedan publicar contenido o realizar cambios en su configuración personal.

```
/**
 * Filtro que se ejecuta en cada petición para verificar si el usuario tiene un token válido
 */
@Component
public class JwtFilter extends OncePerRequestFilter {

    @Autowired
    private JwtUtils jwtUtils;

    @Override
    protected void doFilterInternal(HttpServletRequest request, HttpServletResponse response, FilterChain filterChain)
        throws ServletException, IOException {

        // Buscamos el encabezado "Authorization" en la petición HTTP
        String authHeader = request.getHeader("Authorization");

        // Si el encabezado existe y empieza por "Bearer ", extraemos el token
        if (authHeader != null && authHeader.startsWith("Bearer ")) {
            String token = authHeader.substring(7);

            // Si el token es válido, identificamos al usuario y lo "logueamos" en el contexto de Spring
            if (jwtUtils.validarToken(token)) {
                String usuario = jwtUtils.getUsuarioDesdeToken(token);

                // Creamos un objeto de autenticación para que Spring sepa quién es el usuario
                UsernamePasswordAuthenticationToken auth =
                    new UsernamePasswordAuthenticationToken(usuario, null, new ArrayList<>());

                SecurityContextHolder.getContext().setAuthentication(auth);
            }
        }

        // Dejamos que la petición siga su camino hacia el controlador correspondiente
        filterChain.doFilter(request, response);
    }
}
```

5. Interfaz de Terminal Financiera (TradingView)

En el frontend, se ha integrado la tecnología de TradingView para ofrecer una experiencia de análisis profesional. Mediante un componente de React, se inyecta de forma dinámica el motor gráfico en el DOM, permitiendo que el usuario interactúe con el mercado en tiempo real, utilice indicadores técnicos y visualice velas japonesas sin salir de la plataforma.

```
/**
 * Componente que muestra un gráfico avanzado de TradingView
 */
const TradingViewChart = ({ symbol, exchange }) => {
  const containerId = "tv_chart_main_container";
  const widgetRef = useRef(null);

  useEffect(() => {
    const initWidget = () => {
      if (typeof window.TradingView !== 'undefined' && document.getElementById(containerId)) {
        let formattedSymbol = symbol;

        // Mapeo manual de símbolos para que coincidan las etiquetas de Yahoo con las de TradingView
        const symbolMapping = {
          '^GSPC': 'SPX',
          '^IBEX': 'IBEX35',
          '^IXIC': 'IXIC',
          '^DJI': 'DJI',
          '^GDAXI': 'DAX',
          '^FCHI': 'PX1',
          'GC=F': 'GOLD',
          'CL=F': 'USOIL',
          'SI=F': 'SILVER'
        };

        // Lógica para formatear el símbolo (ej: añadir el mercado como NASDAQ: o BINANCE:)
        if (symbolMapping[symbol]) {
          formattedSymbol = symbolMapping[symbol];
        } else if (!symbol.includes(':')) {
          if (exchange === 'NMS' || exchange === 'NASDAQ') {
            formattedSymbol = `NASDAQ:${symbol}`;
          } else if (exchange === 'NYSE' || exchange === 'NYSE') {
            formattedSymbol = `NYSE:${symbol}`;
          } else if (exchange === 'CCC' || symbol.includes('-')) {
            const cleanSym = symbol.replace('-USD', '').replace('-', '');
            formattedSymbol = `BINANCE:${cleanSym}USD`;
          } else if (exchange === 'CCY') {
            const cleanForex = symbol.replace('X', '');
            formattedSymbol = `FX:${cleanForex}`;
          }
        }

        // Si el símbolo es el mismo que ya está cargado, no hacemos nada para evitar parpadeos
        if (widgetRef.current && widgetRef.current.options?.symbol === formattedSymbol) {
          return;
        }

        // Configuración visual del widget (colores, idioma, herramientas)
        widgetRef.current = new window.TradingView.widget({
          "autosize": true,
          "symbol": formattedSymbol,
          "interval": "D",
          "timezone": "Etc/UTC",
          "theme": "dark",
          "style": "1",
          "locale": "es",
          "toolbar_bg": "#0f172a",
          "enable_publishing": false,
          "withdateranges": true,
          "hide_side_toolbar": true, // Ocultamos herramientas de dibujo para que el gráfico esté limpio
          "allow_symbol_change": false,
          "container_id": containerId,
          "save_image": false,
        });
      }
    };

    initWidget();
  }, [symbol, exchange]);
};
```

6. Sistema de Notificaciones e Interacción Social

La plataforma cuenta con un motor de eventos diseñado para fomentar la interacción dentro de la comunidad. Cada vez que ocurre una acción relevante (un nuevo seguidor, una reacción a un análisis o una publicación de un analista seguido), el sistema genera automáticamente una notificación personalizada. Esta lógica está integrada mediante transacciones atómicas que aseguran que, si la acción principal se realiza con éxito, el aviso llegue correctamente al destinatario, permitiendo una experiencia social dinámica y conectada.

```
// Permite a los usuarios dar o quitar un voto a un análisis
@PostMapping("/{id}/votar")
public ResponseEntity<> votarAnalisis(@PathVariable Long id, Principal principal) {
    try {
        if (principal == null) return ResponseEntity.status(401).body("No autorizado");
        analisisService.alternarVoto(id, principal.getName());
        return ResponseEntity.ok().build();
    } catch (Exception e) {
        return ResponseEntity.badRequest().body(e.getMessage());
    }
}
```

```
// Añade o quita un voto (like) de un usuario a una publicación
@Transactional
public void alternarVoto(Long analisisId, String username) {
    Analisis analisis = obtenerPorId(analisisId);
    Usuario usuario = userService.obtenerPorNombre(username);

    Optional<Voto> votoExistente = votoRepository.findByUsuarioAndAnalisis(usuario, analisis);

    if (votoExistente.isPresent()) {
        votoRepository.delete(votoExistente.get()); // Si ya existía, lo quitamos (unlike)
    } else {
        Voto nuevoVoto = new Voto();
        nuevoVoto.setUsuario(usuario);
        nuevoVoto.setAnalisis(analisis);
        votoRepository.save(nuevoVoto); // Si no existía, lo añadimos (like)

        // Notificar al autor del análisis
        Usuario autor = analisis.getUsuario();
        // Solo notificamos si el que da el like no es el propio autor
        if (autor != null && !autor.getUsuario().equals(username)) {
            // Si es nulo, asumimos que sí quiere notificaciones por defecto
            boolean quiereNotis = autor.getNotificarReacciones() == null || autor.getNotificarReacciones();
            if (quiereNotis) {
                notificacionService.crearNotificacion(
                    autor,
                    "@" + username + " ha reaccionado a tu análisis sobre " + analisis.getActivo().getNombre(),
                    "/comunidad?analisis=" + analisis.getIdentificador(),
                    usuario
                );
            }
        }
    }
}
```

8. Pruebas y validación

8.1. Pruebas unitarias

Debido a la naturaleza individual del proyecto y al enfoque centrado en la agilidad del desarrollo y la integración, no se ha optado por la creación de baterías de pruebas unitarias automatizadas (como JUnit). En su lugar, se ha seguido un proceso de verificación unitaria manual durante la fase de codificación de cada módulo.

Este proceso ha consistido en aislar cada funcionalidad crítica antes de integrarla en el flujo general de la aplicación, utilizando herramientas como Postman para interactuar directamente con los endpoints del backend. Mediante esta técnica, se han validado de forma independiente los siguientes componentes:

- **Lógica de autenticación:** Verificación de la correcta generación de tokens JWT tras el inicio de sesión con credenciales válidas y la denegación de acceso con credenciales erróneas.
- **Servicio de Datos Financieros:** Validación de que la API de Yahoo Finance devuelve los precios en el formato esperado y que el sistema de caché interno funciona correctamente.
- **Cálculo del Ranking:** Pruebas controladas con datos simulados para asegurar que el algoritmo de éxito calcula los porcentajes de acierto de forma precisa basándose en el precio objetivo.
- **Gestión de archivos en S3:** Comprobación de que el servicio de subida a la nube devuelve la URL correcta de la imagen y maneja adecuadamente los posibles errores de red con AWS.

Esta metodología de verificación constante ha permitido asegurar que las funciones individuales del sistema fueran sólidas antes de pasar a las fases de integración y pruebas funcionales.

8.2. Pruebas de integración

Una vez verificados los componentes individuales, se procedió a realizar pruebas de integración para asegurar que la comunicación entre los distintos subsistemas de SocialMarkets fuera fluida y libre de errores. Estas pruebas se centraron en validar el flujo de información de extremo a extremo en los siguientes escenarios:

- **Integración Frontend-Backend:** Se validó que el cliente web desarrollado en React consumiera correctamente la API REST de Spring Boot. Se comprobó el manejo de estados de carga, la recepción de datos en formato JSON y el procesamiento de códigos de error HTTP (como el 401 para sesiones caducadas o el 404 para recursos no encontrados).
- **Ciclo de Persistencia en la Nube (AWS S3):** Se realizaron pruebas conjuntas para asegurar que, al subir una imagen desde el frontend, el backend la procesara correctamente, la almacenara en el bucket de S3 y, finalmente, guardara la URL resultante en la base de datos PostgreSQL. Esta prueba confirmó la correcta sincronización entre tres sistemas distintos.
- **Sincronización con APIs Financieras:** Se verificó que los datos obtenidos de Yahoo Finance alimentaran correctamente la lógica interna del ranking. Se comprobó que el flujo "Solicitud de precio -> Respuesta de API externa -> Cálculo de acierto -> Actualización en DB" se completara de forma correcta.
- **Flujo de Seguridad con JWT:** Se validó la persistencia del token de seguridad en el navegador del usuario y su correcta inclusión en las cabeceras de cada petición asíncrona, asegurando que el backend identificara correctamente al usuario en cada interacción.

El éxito de estas pruebas de integración garantizó que la arquitectura del proyecto fuera capaz de funcionar en conjunto.

9. Despliegue y mantenimiento

9.1. Despliegue de la aplicación

El despliegue de SocialMarkets se ha realizado siguiendo un modelo de arquitectura híbrida, donde la lógica de aplicación se ejecuta en el entorno local de demostración y la persistencia de datos se apoya en la infraestructura de AWS Academy. Los pasos realizados para la puesta en marcha del sistema son los siguientes:

1. Configuración de la base de datos en AWS (RDS/Laboratorio): Al utilizar el laboratorio de AWS Academy, se ha desplegado una instancia de **PostgreSQL 16**. Los pasos técnicos han sido:

- **Apertura de puertos:** Configuración del Security Group en la consola de AWS para permitir el tráfico entrante a través del puerto **5432** desde la dirección IP de la máquina local.
- **Inicialización:** Creación de la base de datos socialmarkets y configuración de las credenciales de acceso (usuario maestro y contraseña).

2. Preparación del entorno de ejecución local: Se ha verificado que el equipo de demostración cuente con el software base necesario para la ejecución de los servicios:

- **Java Development Kit (JDK) 25:** Necesario para compilar y ejecutar el backend de Spring Boot.
- **Node.js (v20 o superior):** Requerido para gestionar las dependencias del frontend y ejecutar el servidor de desarrollo de Vite.
- **Maven:** Para la gestión del ciclo de vida del proyecto Java.

3. Configuración de credenciales y variables de entorno: Antes de la ejecución, se han configurado las propiedades de conexión en el archivo `application.properties` del backend:

- **URL de conexión:**

jdbc:postgresql://[ENDPOINT_AWS]:5432/socialmarkets

- **Credenciales de AWS S3:** Inserción de la Access Key y Secret Key proporcionadas por el laboratorio para permitir la subida de imágenes de perfil al bucket de S3.

4. Despliegue del Backend (Spring Boot): Desde la raíz del módulo backend-core, se ejecutan los siguientes comandos para poner en marcha la API, o se ejecuta desde un IDE como Eclipse:

- mvn clean package: Para compilar el proyecto y generar el archivo ejecutable.
- mvn spring-boot:run: Para levantar el servidor en el puerto **8080**, inicializando el pool de conexiones con AWS.

5. Despliegue del Frontend (React/Vite): Una vez que el backend está operativo, se procede con el despliegue de la interfaz de usuario:

- npm install: Para descargar todas las librerías necesarias (Framer Motion, Axios, Lucide, etc.).
- npm run dev: Para iniciar el servidor web en el puerto **5173**.

La aplicación será accesible a través de la dirección **<http://localhost:5173>**, comunicándose de forma transparente con los servicios en la nube de Amazon Web Services.

10. Conclusiones y trabajo futuro

10.1. Conclusiones

Tras la finalización del desarrollo de SocialMarkets, la conclusión principal es que se han cumplido con éxito todos los objetivos generales y específicos planteados al inicio del proyecto. Se ha logrado construir una plataforma funcional que no solo ofrece herramientas técnicas de análisis financiero, sino que establece un sistema de reputación basado en méritos reales, cumpliendo así con la propuesta de valor diferenciadora del proyecto.

El desarrollo de este TFG ha supuesto un aprendizaje en el uso de arquitecturas nuevas. La integración de tecnologías como Spring Boot y React ha permitido comprender profundamente el flujo de comunicación cliente-servidor y la importancia de la seguridad mediante JWT. Uno de los mayores retos ha sido la gestión de datos externos en tiempo real, trabajar con las APIs de Yahoo Finance y TradingView (que exigió un diseño de la lógica de negocio para asegurar que la información fuera precisa y que el sistema de ranking respondiera de forma coherente a los movimientos del mercado).

A pesar del cumplimiento de los objetivos, la principal limitación ha sido el tiempo y el alcance de la infraestructura. Se tomó la decisión estratégica de centrar los esfuerzos en la lógica financiera y en la calidad de la interfaz de usuario, optando por un despliegue híbrido para la demostración. Esta decisión ha permitido entregar un producto pulido y funcional, priorizando la estabilidad del motor de cálculo y la experiencia del usuario sobre la complejidad de una orquestación de servidores más pesada.

10.2. Trabajo futuro

SocialMarkets es un proyecto con un gran potencial de crecimiento. Si se dispusiese de más tiempo y recursos, se han identificado las siguientes líneas de mejora y expansión:

- **Despliegue integral y productivo:** La principal evolución natural del proyecto sería la dockerización completa de los servicios para facilitar un despliegue escalable en la nube (AWS EC2), permitiendo el acceso a la plataforma mediante una URL pública y estable.
- **Versión móvil:** Dado el carácter social de la aplicación, el desarrollo de una versión móvil utilizando React Native permitiría a los analistas recibir notificaciones push y consultar el mercado de forma más ágil desde cualquier lugar.
- **Funcionalidades sociales avanzadas:** Implementar un sistema de comentarios en las publicaciones, chats privados entre analistas y la posibilidad de crear grupos de inversión temáticos dentro de la comunidad.
- **Integración con Brókers:** Permitir que los usuarios, además de analizar y compartir ideas, puedan ejecutar operaciones directamente desde la plataforma conectando sus cuentas de bróker externas.
- **IA para análisis predictivo:** Incorporar modelos de Inteligencia Artificial que ayuden a los usuarios a identificar patrones técnicos o que generen alertas automáticas basadas en el sentimiento de la comunidad.

11. Relación del proyecto con los módulos del ciclo

El desarrollo de SocialMarkets ha permitido integrar y poner en práctica de forma transversal los conocimientos adquiridos en los distintos módulos del Ciclo Formativo de Grado Superior. A continuación, se detalla dicha relación:

- **Desarrollo de Interfaces:** Este módulo ha sido fundamental para la creación del frontend. Se han aplicado principios de diseño centrados en el usuario (UX) para crear una interfaz intuitiva y profesional.
 - Diseño de prototipos de alta fidelidad.
 - Implementación de una interfaz reactiva mediante la librería React.
 - Uso de CSS avanzado para lograr una estética premium (Glassmorphism).
- **Acceso a Datos:** Ha sido la base para la construcción de la lógica del servidor y la persistencia de la información.
 - Uso del framework Spring Boot para la creación de servicios.
 - Mapeo objeto-relacional mediante Hibernate y Spring Data JPA.
 - Conectividad y gestión de transacciones con la base de datos remota.
- **Bases de Datos:** Fundamental para la estructuración de la información financiera y de comunidad.
 - Diseño y normalización del Modelo Entidad-Relación (Usuarios, Análisis, Votos).
 - Implementación de la base de datos en PostgreSQL.
 - Optimización de consultas SQL y gestión de claves foráneas para mantener la integridad de los datos.
- **Programación de Servicios y Procesos:** Módulo fundamental para la comunicación y seguridad de la plataforma (aunque no se ha llegado a tratar en el curso).
 - Implementación de una API REST para la comunicación cliente-servidor.
 - Gestión de la seguridad y autenticación mediante JSON Web Tokens (JWT).

- Consumo de servicios web externos y APIs de terceros (Yahoo Finance).
- Programación de procesos asíncronos para la actualización de precios.
- **Entornos de Desarrollo:** Aplicado en la gestión profesional del ciclo de vida del software.
 - Uso de Git y GitHub para el control de versiones y trazabilidad del código.
 - Utilización de IDEs como Eclipse y VS Code.
 - Planificación y seguimiento de tareas durante todas las fases del proyecto.
- **Programación:** La base sobre la que se asienta toda la lógica del backend.
 - Aplicación de los principios de Programación Orientada a Objetos (POO) en Java.
 - Desarrollo de algoritmos para el motor de cálculo del ranking de analistas.
 - Gestión de excepciones y estructuras de datos complejas.

12. Bibliografía/Webgrafía

Documentación de Tecnologías y Frameworks

- **Spring Boot:** Documentación oficial del framework para el desarrollo del backend. <https://spring.io/projects/spring-boot>
- **React:** Guía de referencia oficial para la construcción de interfaces basadas en componentes. <https://react.dev/>
- **PostgreSQL:** Manual técnico de referencia del sistema gestor de bases de datos. <https://www.postgresql.org/docs/>
- **AWS S3 SDK for Java:** Guía de integración de servicios de almacenamiento en la nube de Amazon. <https://docs.aws.amazon.com/sdk-for-java/>

APIs e Integraciones Externas

- **TradingView Widgets:** Documentación técnica para la integración de gráficos y terminales financieros. <https://www.tradingview.com/widget/>
- **Yahoo Finance API (Community Documentation):** Referencia para el consumo de datos de mercado en tiempo real. <https://query1.finance.yahoo.com/>
- **Framer Motion:** Biblioteca de animaciones para React utilizada en el desarrollo de la UI. <https://www.framer.com/motion/>

Herramientas de Desarrollo y Soporte

- **Postman:** Herramienta para el testeo y validación de APIs REST. <https://www.postman.com/>
- **Spring Initializer:** Generador de estructura base para proyectos Spring Boot. <https://start.spring.io/>
- **MDN Web Docs:** Recurso de referencia para estándares de JavaScript, HTML5 y CSS3. <https://developer.mozilla.org/>
- **Modelos de Lenguaje (IA):** <https://gemini.google.com/>, <https://chatgpt.com/>, <https://chat.deepseek.com/>